



US 20210399898A1

(19) **United States**(12) **Patent Application Publication**
Wright et al.(10) **Pub. No.: US 2021/0399898 A1**(43) **Pub. Date: Dec. 23, 2021**(54) **COMPUTER IMPLEMENTED SYSTEM AND
METHOD FOR STORING DATA ON A
BLOCKCHAIN**Nov. 27, 2018 (GB) 1819293.0
Nov. 27, 2018 (GB) 1819297.1
Nov. 27, 2018 (GB) 1819299.7(71) Applicant: **nChain Holdings Limited**, St. John's
(AG)**Publication Classification**(72) Inventors: **Craig Steven Wright**, London (GB);
Jack Owen Davies, London (GB);
Chloe Ceren Tartan, London (GB);
Owen Vaughan, London (GB)(51) **Int. Cl.**
H04L 9/32 (2006.01)
G06F 16/23 (2006.01)
H04L 9/30 (2006.01)(52) **U.S. Cl.**
CPC **H04L 9/3247** (2013.01); **G06F 16/2379**
(2019.01); **H04L 2209/38** (2013.01); **H04L**
9/3239 (2013.01); **H04L 9/3073** (2013.01)(21) Appl. No.: **17/297,338**(22) PCT Filed: **Nov. 14, 2019**(86) PCT No.: **PCT/IB2019/059808**

§ 371 (c)(1),

(2) Date: **May 26, 2021**(57) **ABSTRACT**

A method of storing data on a blockchain, such as the bitcoin blockchain, is disclosed. The method comprises generating a blockchain transaction having at least one first input and/or at least one first output containing first data (<Content 1>) to be stored on the blockchain, and at least one second input and/or at least one second output containing second data (<Attribute 1>,<Attribute 2>) representing at least one attribute of the first data. At least one second input and/or at least one second output is separate from the or each first input and/or first output.

(30) **Foreign Application Priority Data**Nov. 27, 2018 (GB) 1819284.9
Nov. 27, 2018 (GB) 1819286.4
Nov. 27, 2018 (GB) 1819290.6
Nov. 27, 2018 (GB) 1819291.4

<i>TxID</i>	
Inputs	Outputs
<Sig P> <P>	OP_RETURN <Metanet Flag> <Attribute 1> <Attribute 2>
	<Content 1> OP_DROP <H(P')> [CheckSig P']

<i>TxID</i>	
Inputs	Outputs
< Sig P> <P>	OP_RETURN <Metanet Flag> <Attribute 1> <Attribute 2>
	<Content 1> OP_DROP <H(P')> [CheckSig P']

Fig. 1

<i>TxID</i>	
Inputs	Outputs
< Sig P> <P> <Content 1> OP_DROP	OP_RETURN <Metanet Flag> <Attribute 1> <Attribute 2>

Fig. 2

$TxID_1$	
Inputs	Outputs
$\langle Sig\ P \rangle \langle P \rangle$	OP_RETURN $\langle Metanet\ Flag \rangle \langle Attribute\ 1,1 = Content\ name \rangle$ $\langle Attribute\ 1,2 = Recombination\ scheme \rangle$ $\langle Attribute\ 1,3 = Content\ chunk\ index\ 1 \rangle$
	$\langle Content\ chunk\ 1 \rangle OP_DROP\ \langle H(P') \rangle [CheckSig\ P']$

$TxID_2$	
Inputs	Outputs
$\langle Sig\ P \rangle \langle P \rangle$	OP_RETURN $\langle Metanet\ Flag \rangle \langle Attribute\ 2,1 = Content\ name \rangle$ $\langle Attribute\ 2,2 = Recombination\ scheme \rangle$ $\langle Attribute\ 2,3 = Content\ chunk\ index\ 2 \rangle$
	$\langle Content\ chunk\ 2 \rangle OP_DROP\ \langle H(P') \rangle [CheckSig\ P']$

Fig. 3

$TxID_{Bob}$			
Inputs		Outputs	
Value	Script	Value	Script
x	$\langle Sig\ P_B \rangle \langle P_B \rangle$	x	$[Private\ Key\ Puzzle\ P_1, r_0] [CheckSig\ P_A]$

Fig. 4

$TxID_{Alice}$			
Inputs		Outputs	
Value	Script	Value	Script
x	$\langle Sig P_A \rangle \langle P_A \rangle \langle Sig P_1, r_0 \rangle \langle P_1 \rangle$	x	$[CheckSig P_A]$

Fig. 5

Use		Alice	Bob
<i>Token Issuance</i>	Purchase of 10 tokens	$I_{Alice} = k$	$I_{Bob} = H^{10}(Y)$
<i>Token redemption</i>	Redeem token 1	X_1	$T_1 = H^9(Y)$
	Redeem token 2	X_2	$T_2 = H^8(Y)$
	...	...	...
	Redeem token 10	X_{10}	$T_{10} = Y$

Fig. 6

$TxID_{Alice}$			
Inputs		Outputs	
Value	Script	Value	Script
x	$\langle Sig P_A \rangle \langle P_A \rangle$	x	$[Hash\ Puzzle\ H(I_{Alice})] [Hash\ Puzzle\ H(I_{Bob})] [CheckSig\ P_B]$

Phase 1.1 (Alice to Bob):

Fig. 7

$TxID_{Bob}$			
Inputs		Outputs	
Value	Script	Value	Script
$10 + x$	$\langle Sig P_B \rangle \langle P_B \rangle$	$10 + x$	$[Hash\ Puzzle\ H(I_{Alice})] [Hash\ Puzzle\ H(I_{Bob})] [CheckSig\ P_A]$

Phase 1.2 (Bob to Alice):

Fig. 8

Phase 2.1 (Bob to Alice):

$TxID_{Bob}$			
Inputs		Outputs	
Value	Script	Value	Script
x	$\langle Sig P_B \rangle \langle P_B \rangle$	x	$[Hash\ Puzzle\ H(X_1)] [Hash\ Puzzle\ H(T_1)] [CheckSig\ P_A]$

Fig. 9

Phase 2.2 (Alice to Bob):

$TxID_{Alice}$			
Inputs		Outputs	
Value	Script	Value	Script
x	$\langle Sig P_A \rangle \langle P_A \rangle$	x	$[Hash\ Puzzle\ H(X_1)] [Hash\ Puzzle\ H(T_1)] [CheckSig\ P_B]$

Fig. 10

Phase 2.3 (Alice to Alice):

$TxID_{Alice}$			
Inputs		Outputs	
Value	Script	Value	Script
x	$\langle Sig P_A \rangle \langle P_A \rangle \langle T_1 \rangle \langle X_1 \rangle$	x	$[CheckSig P_A]$

Fig. 11

Phase 2.4 (Bob to Bob):

$TxID_{Bob}$			
Inputs		Outputs	
Value	Script	Value	Script
x	$\langle Sig P_B \rangle \langle P_B \rangle \langle T_1 \rangle \langle X_1 \rangle$	x	$[CheckSig P_B]$

Fig. 12

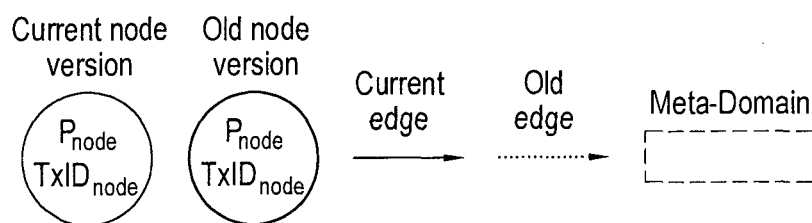
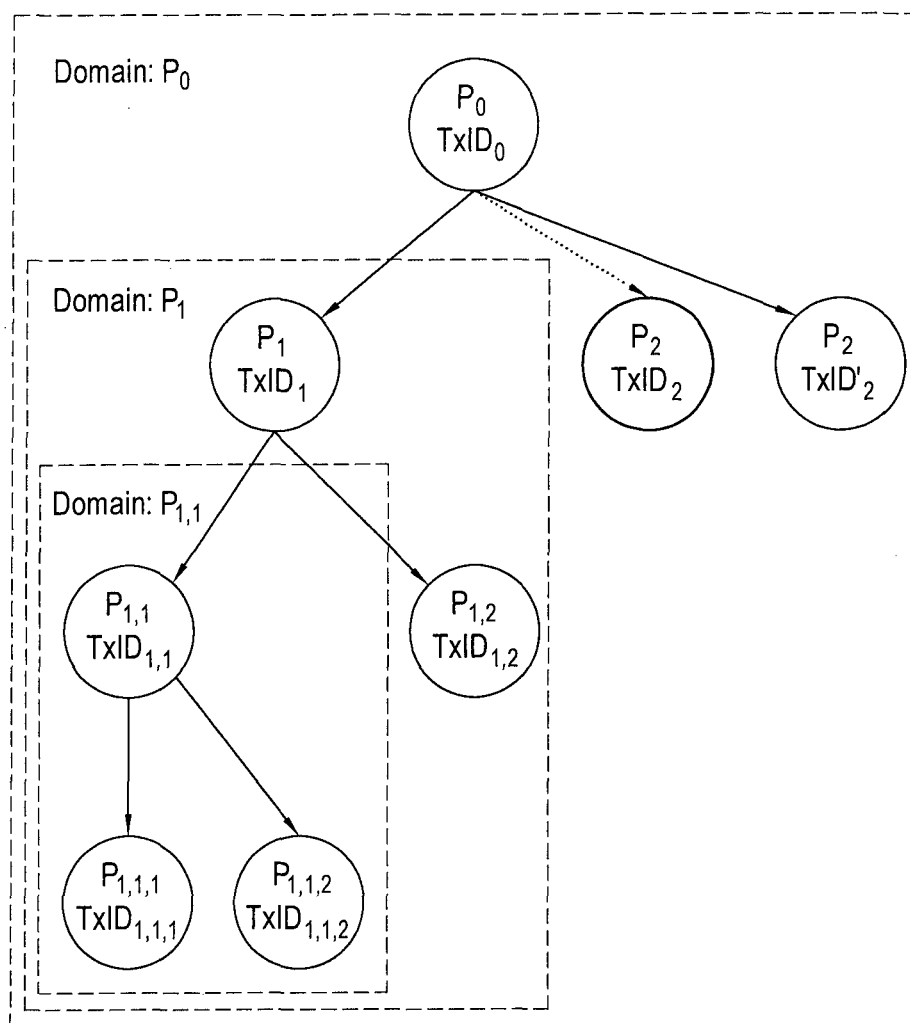


Fig. 13

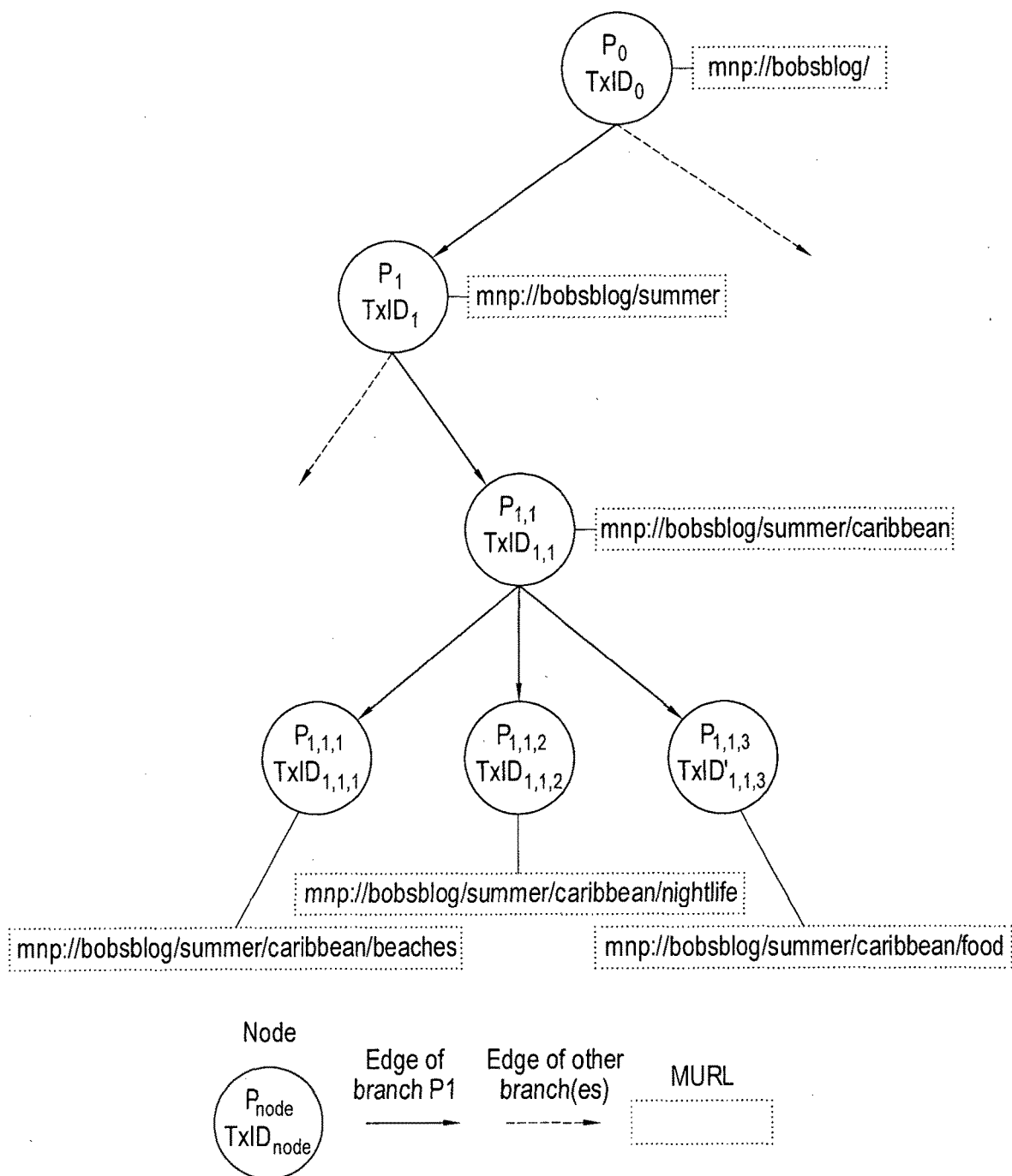


Fig. 14

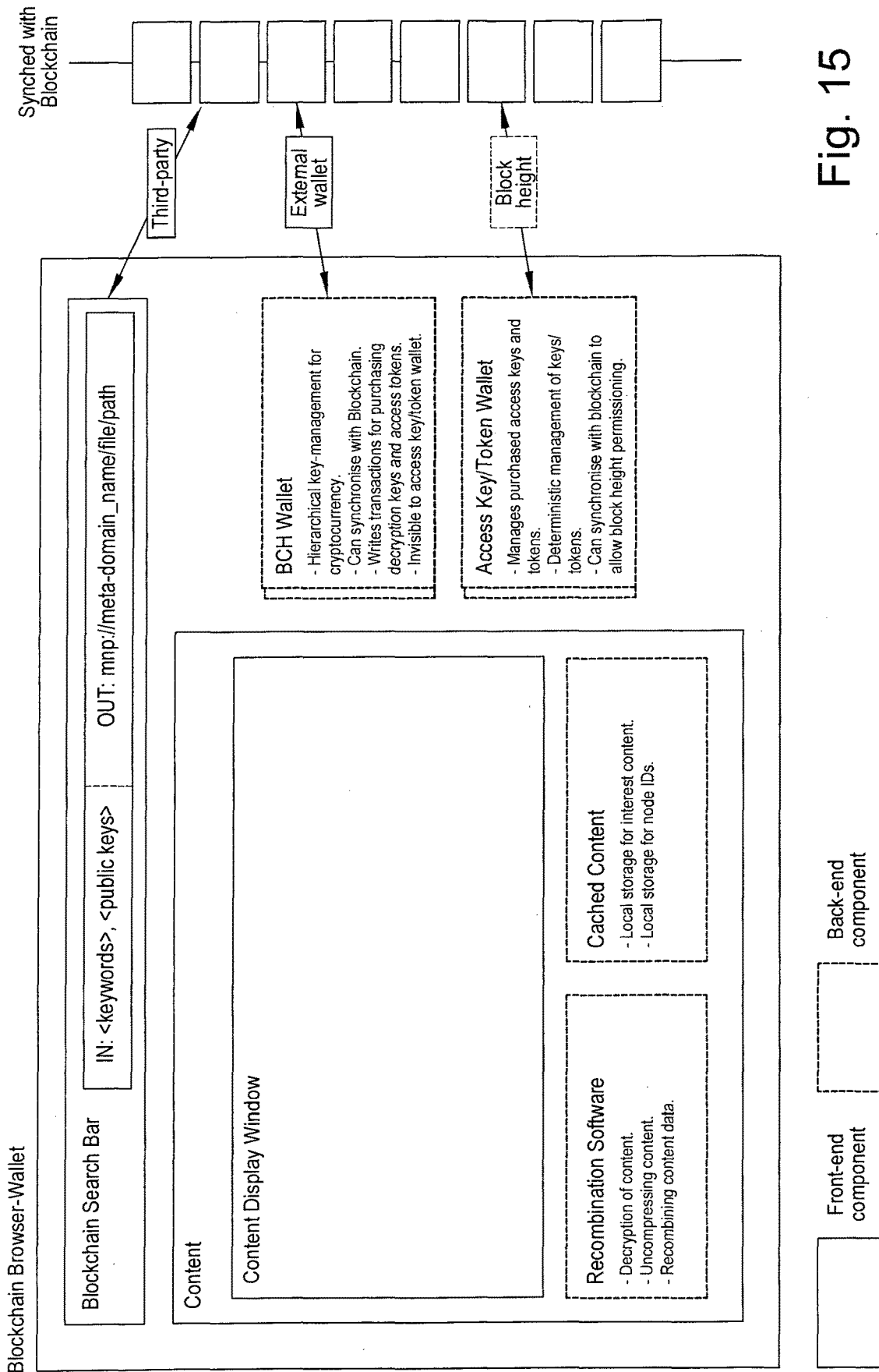


Fig. 15

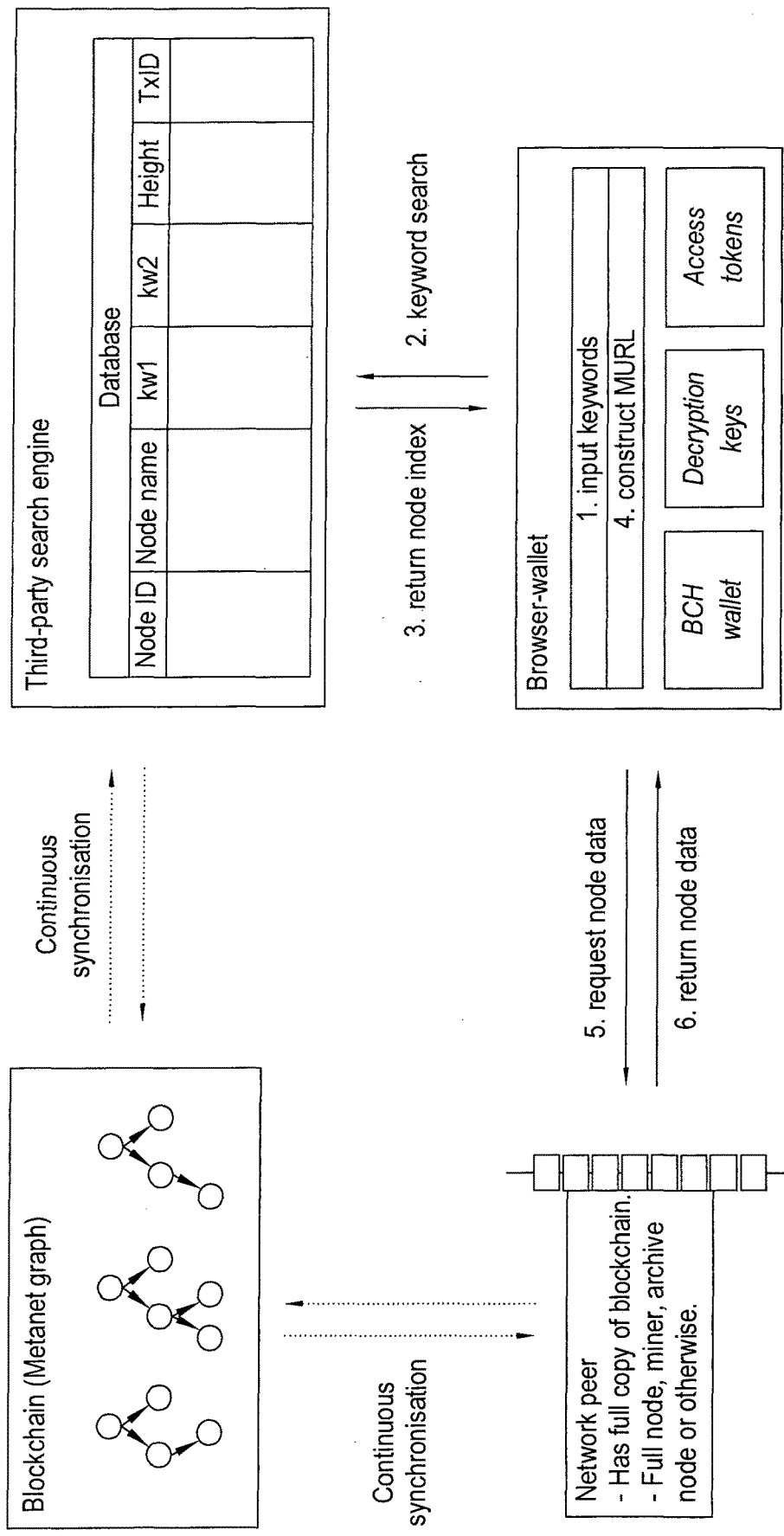


Fig. 16

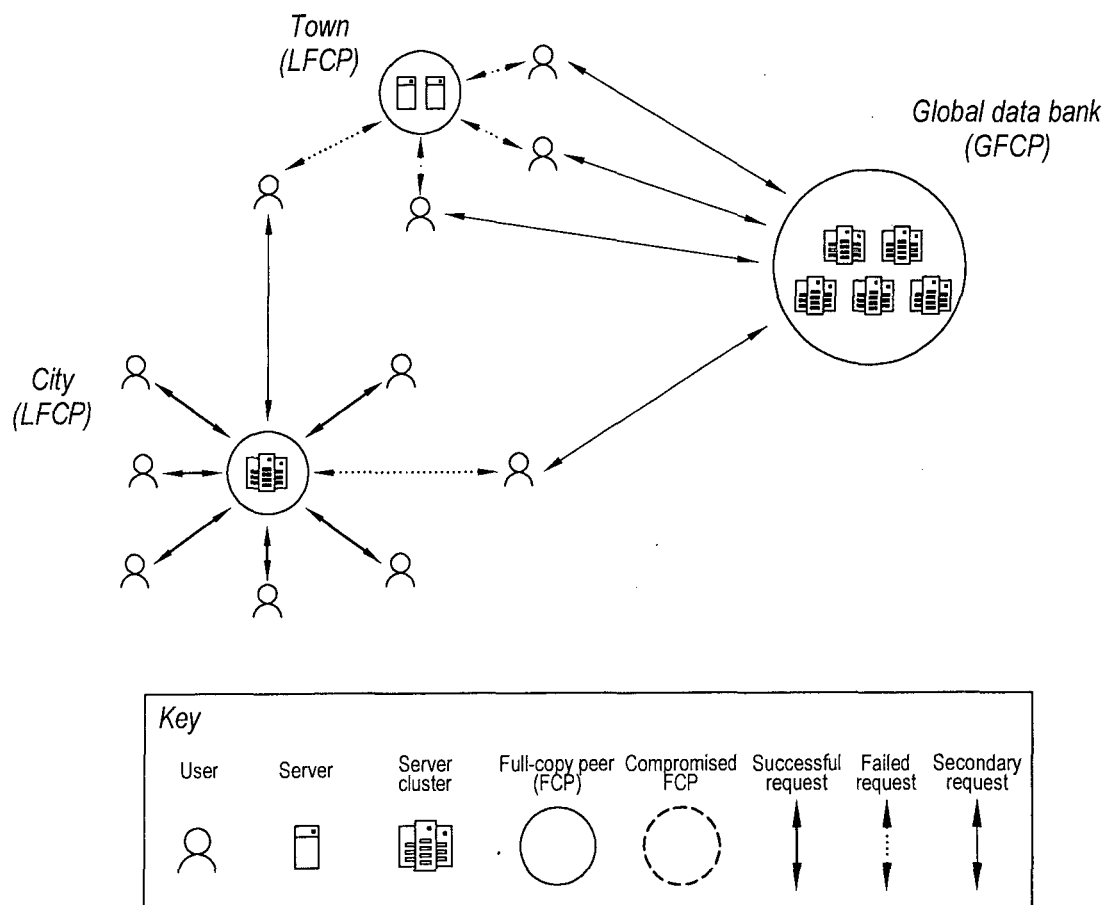


Fig. 17

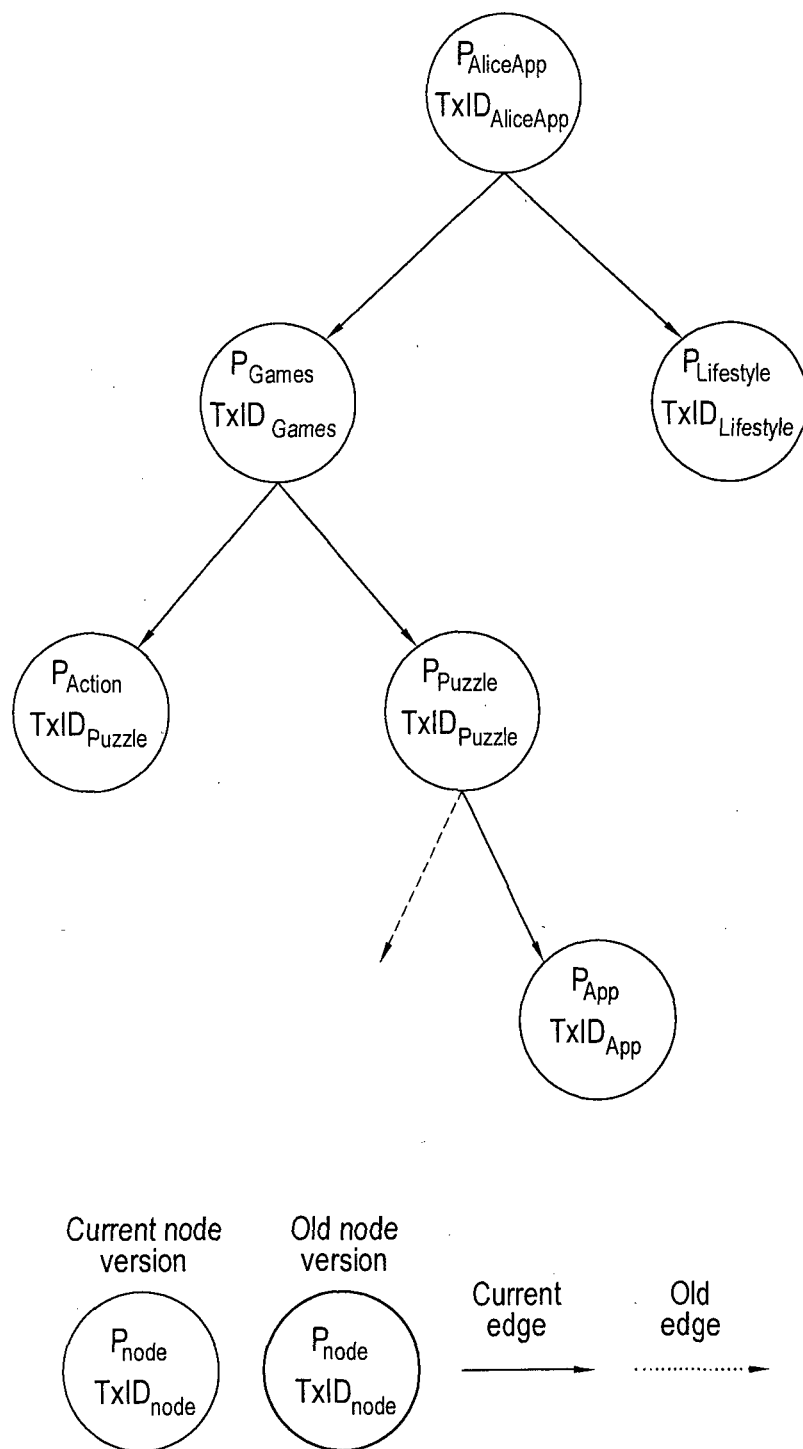


Fig. 18

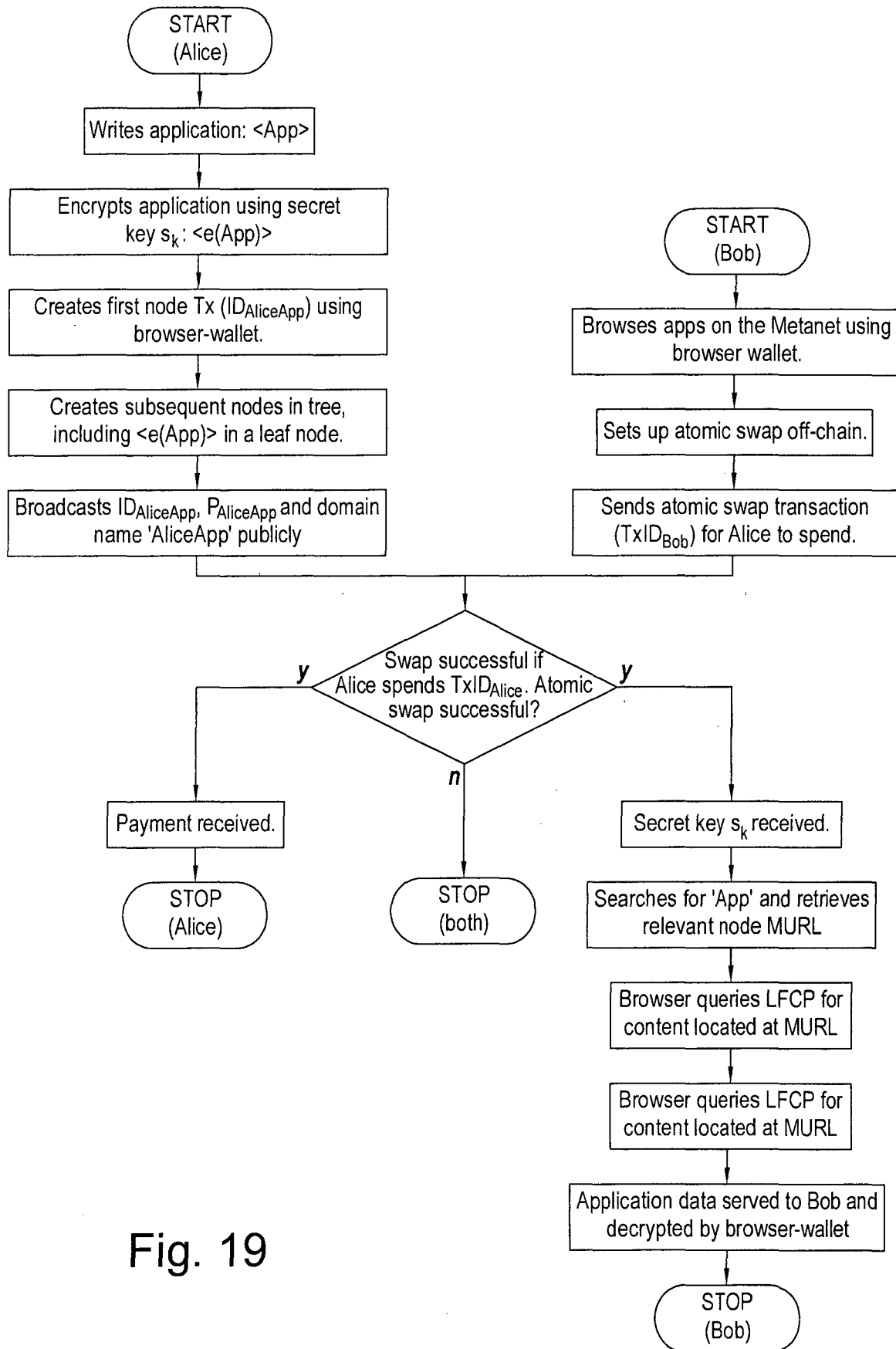


Fig. 19

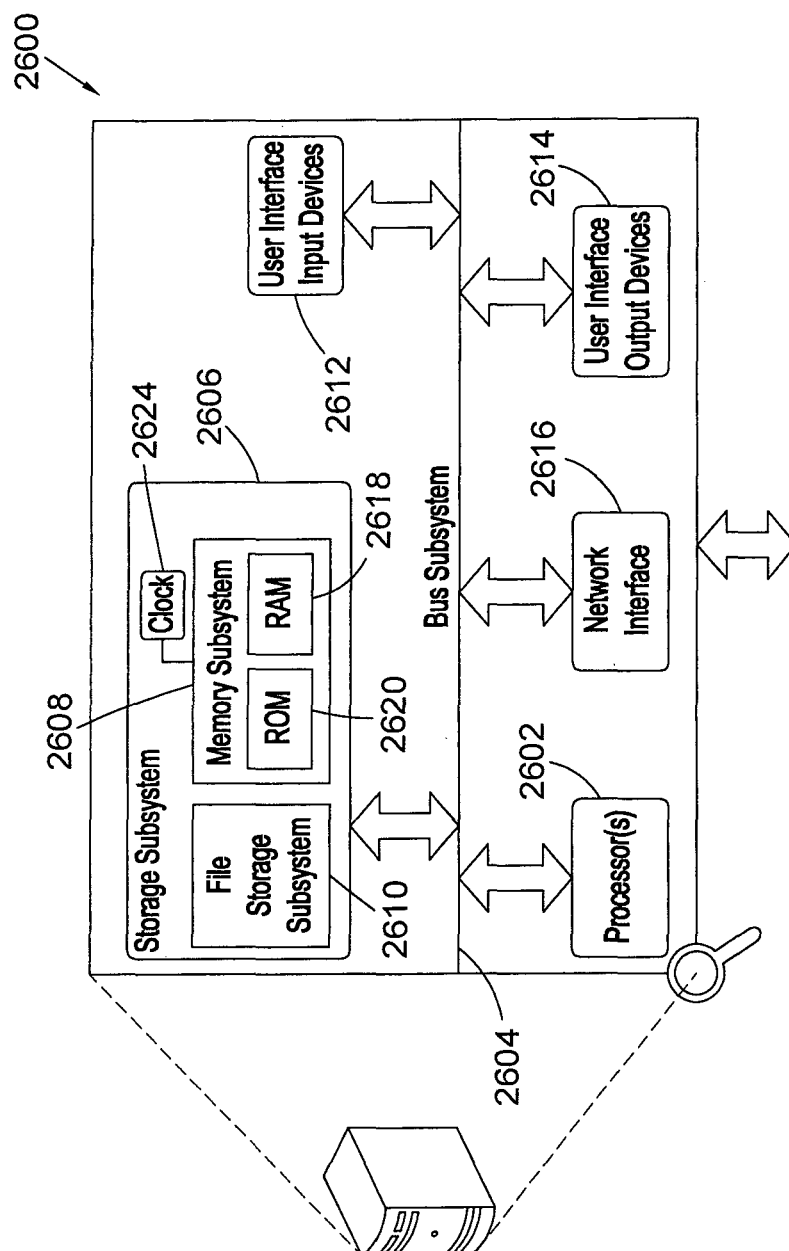


Fig. 20

COMPUTER IMPLEMENTED SYSTEM AND METHOD FOR STORING DATA ON A BLOCKCHAIN

[0001] This invention relates generally to cryptographic techniques for improved data communication and exchange across an electronic network, and in particular a peer-to-peer network such as a blockchain network. It relates to data storage, access, retrieval and processing, and more particularly to such data-related activities on a blockchain. The invention is particularly suited, but not limited to, use in processing data in a manner similar to that provided by web sites and web pages but using the blockchain as an underlying mechanism or platform rather than web server(s). Thus, the invention provides a secure, efficient, cryptographically-enforced, alternative infrastructure for data processing and transfer.

[0002] In this document we use the term ‘blockchain’ to include all forms of electronic, computer-based, distributed ledgers. These include consensus-based blockchain and transaction-chain technologies, permissioned and unpermissioned ledgers, shared ledgers and variations thereof. The most widely known application of blockchain technology is the Bitcoin ledger, although other blockchain implementations have been proposed and developed. While Bitcoin may be referred to herein for the purpose of convenience and illustration, it should be noted that the invention is not limited to use with the Bitcoin blockchain and alternative blockchain implementations and protocols fall within the scope of the present invention. The term “user” may refer herein to a human or a processor-based resource. “Bitcoin” as used herein includes all versions and variations of protocols which derive from the Bitcoin protocol.

[0003] A blockchain is a peer-to-peer, electronic ledger which is implemented as a computer-based decentralised, distributed system made up of blocks which in turn are made up of transactions. Each transaction is a data structure that encodes the transfer of control of a digital asset between participants in the blockchain system, and includes at least one input and at least one output. Each block contains a hash of the previous block so that blocks become chained together to create a permanent, unalterable record of all transactions which have been written to the blockchain since its inception. Transactions contain small programs known as scripts embedded into their inputs and outputs, which specify how and by whom the outputs of the transactions can be accessed. On the Bitcoin platform, these scripts are written using a stack-based scripting language.

[0004] In order for a transaction to be written to the blockchain, it must be “validated”. Network nodes (miners) perform work to ensure that each transaction is valid, with invalid transactions rejected from the network. Software clients installed on the nodes perform this validation work on an unspent transaction (UTXO) by executing its locking and unlocking scripts. If execution of the locking and unlocking scripts evaluate to TRUE, the transaction is valid and the transaction is written to the blockchain. Thus, in order for a transaction to be written to the blockchain, it must be i) validated by the first node that receives the transaction—if the transaction is validated, the node relays it to the other nodes in the network; and ii) added to a new block built by a miner; and iii) mined, i.e. added to the public ledger of past transactions.

[0005] Although blockchain technology is most widely known for the use of cryptocurrency implementation, digital entrepreneurs have begun exploring the use of both the cryptographic security system Bitcoin is based on and the data that can be stored on the Blockchain to implement new systems. It would be highly advantageous if the blockchain could be used for tasks and processes which are not limited to the realm of cryptocurrency. Such solutions would be able to harness the benefits of the blockchain (e.g. a permanent, tamper proof records of events, distributed processing etc) while being more versatile in their applications.

[0006] One such area of interest is the use of the blockchain for the storage, sharing, accessing and controlling of data among users. Today, this is achieved via the Internet, with servers hosting web sites and pages which users visit in order to access the desired data, typically with a search engine.

[0007] However, some observers have begun to envisage the use of the blockchain to address some of the disadvantages of the Internet, such as control of large amounts of data and content by centralised parties. See, for example, “Life After Google: The Fall of Big Data and the Rise of the Blockchain Economy”, George Gilder, Gateway Editions, July 2018, ISBN-10: 9781621575764 and ISBN-13: 978-1621575764.

[0008] Thus, it is desirable to provide an arrangement enabling such data to be stored, processed, retrieved, searched and/or shared on the blockchain, advantageously utilising the distributed, unalterable, distributed and permanent nature of the blockchain. Such an improved solution has now been devised.

[0009] Embodiments of the present disclosure provide, at least, alternative, efficient and secure techniques for implementing a blockchain solution and for storing, processing, searching and/or retrieving data thereon or therefrom. Embodiments also provide, at least, an alternative, blockchain-implemented technical infrastructure for storing, processing, retrieving, transferring, searching and/or sharing data between computing nodes. Because the invention enables the blockchain network to be used in a new way and for the provision of an improved and technical result, the invention provides an improved blockchain-implemented network.

[0010] Embodiments also provide solutions for the secure control of access to digital resources over a technically different and improved computing platform which comprises a blockchain and a blockchain protocol.

[0011] The invention is defined in the appended claims.

[0012] In accordance with the invention there may be provided a computer implemented method.

[0013] It may be a method of storing data on a blockchain, the method comprising:

[0014] generating a blockchain transaction having at least one first input and or at least one first output containing first data to be stored on the blockchain, and at least one second input and or at least one second output containing second data representing at least one attribute of said first data, wherein at least one said second input and or at least one said second output is separate from the or each said first input and or first output.

[0015] By storing the first data separately from the second data representing at least one attribute of the first data, this provides the advantage of enabling data to be stored on a

blockchain in a manner independent of the protocol used to process the data, thereby making the process more universally applicable. In addition, separating the first data and the second data provides the advantage of enabling the first data to be more readily digitally signed by a third party. Also, the explicit separation of the data attributes and content provides a standardised procedure for data insertion in the blockchain. This captures both existing data insertion methods as well as offering support for future extensions. In addition, once the data is inserted on the blockchain, it can have a unique identifier and location. This removes the onus on a user to locally store a copy of the data as it is immutable once included on the blockchain. This also means that the data is retrievable by a user at any point in time without the need for any disaster recovery mechanism.

[0016] The first data may be contained in a plurality of said first inputs and or said first outputs.

[0017] By storing the first data in a plurality of first inputs and/or first outputs, this provides the advantage of enabling the density of data storage in the transaction to be maximised.

[0018] Said second data may include data relating to recombination of said first data.

[0019] At least one said second input and or at least one said second output may include indication data indicating that the transaction includes said first data.

[0020] This provides the advantage of avoiding unnecessary processing of blockchain transactions not including the first data, thereby improving efficiency of processing.

[0021] The second data may represent at least one attribute of said first data, being at least one of data type, data encryption scheme, data compression scheme, indexing information, permission information, encoding information, keyword information or searching information of said first data.

[0022] The first data may be contained in at least one said first output.

[0023] This provides the advantage of enabling validity of the transaction to be checked as part of the blockchain mining process. Data integrity is ensured through the existing miner validation process of the underlying blockchain. This allows the blockchain to be used as a trusted global server where the integrity of the stored data is guaranteed by the miners and thus can be stored and distributed by other parties without the need for trust.

[0024] At least one said first and/or second output may include a script opcode for marking an output as invalid for subsequent use as an input to a subsequent transaction.

[0025] This provides the advantage of avoiding processing of the output as part of the unspent transaction output (UTXO) data, thereby improving efficiency of processing.

[0026] At least one said first and/or second output may be spendable.

[0027] This provides the advantage of enabling executable programs to be included in the first data, which could otherwise be prevented from operating in an unspendable output, for example as a result of an OP_RETURN operator. The OP_RETURN operator also has the added advantage of terminating the executable program.

[0028] At least one said first output may be redeemable by means of at least one key corresponding to a respective digital signature applied to said first data.

[0029] This provides the advantage of utilising the process of verifying of digital signatures of the transaction, carried

out as part of the cryptocurrency mining process to verify that the transaction is valid, to further verify validity of the first and/or second data.

[0030] A digital signature may be applied to said first data.

[0031] This provides the advantage of enabling the first data to be more readily isolated from the second data and digitally signed by a third party independently of the second data, thereby making the process more universally applicable, and enabling faster data processing. This also enables the method to be made more protocol-independent.

[0032] At least one said digital signature may be based on a cryptography system having a public-private key pair, wherein the private key is based on a plurality of prime numbers and the corresponding public key is based on a product of a plurality of said prime numbers.

[0033] This provides the advantage of providing signature schemes which are computationally inexpensive to verify, thereby improving efficiency of processing.

[0034] At least one said digital signature may be a Rabin signature.

[0035] This provides the advantage that a distributor can provide authentication from the original source of the data using Rabin signatures. This allows the content itself to be signed for by the original author to prove its authenticity.

[0036] The method may further comprise applying data compression to said first data and/or said second data.

[0037] The invention also provides a system, comprising:

[0038] a processor; and

[0039] memory including executable instructions that, as a result of execution by the processor, causes the system to perform any embodiment of the computer-implemented method described herein.

[0040] The invention also provides a non-transitory computer-readable storage medium having stored thereon executable instructions that, as a result of being executed by a processor of a computer system, cause the computer system to at least perform an embodiment of the computer-implemented method described herein.

[0041] These and other aspects of the present invention will be apparent from and elucidated with reference to, the embodiment described herein. An embodiment of the present invention will now be described, by way of example only, and with reference to the accompanying drawings, in which:

[0042] FIG. 1 shows a blockchain transaction embodying the present invention in which data is stored in a plurality of outputs;

[0043] FIG. 2 shows a blockchain transaction embodying the present invention in which data is stored in an input;

[0044] FIG. 3 shows a series of blockchain transactions embodying the present invention in which data is stored over outputs of a plurality of blockchain transactions;

[0045] FIG. 4 shows a blockchain transaction embodying the present invention transferring cryptocurrency payment to allow access to data by means of an atomic swap;

[0046] FIG. 5 shows a blockchain transaction embodying the present invention for redeeming the payment of the transaction of FIG. 4;

[0047] FIG. 6 shows secret values kept by participants in a blockchain transaction embodying the present invention issuing tokens to allow access to data by means of an atomic swap;

[0048] FIGS. 7 and 8 show blockchain transactions embodying the present invention for issuing tokens to allow access to data by means of an atomic swap;

[0049] FIGS. 9 and 10 show blockchain transactions embodying the present invention for redeeming tokens issued by means of the transactions of FIGS. 7 and 8;

[0050] FIGS. 11 and 12 show blockchain transactions for accessing the secrets exchanged by the transactions of FIGS. 9 and 10;

[0051] FIG. 13 provides an illustration of the Metanet graph structure in accordance with an embodiment of the present invention.

[0052] FIG. 14 shows an illustration of a Metanet graph tree for the domain 'bobsblog' including MURL search paths in accordance with an embodiment of the present invention.

[0053] FIG. 15 shows a schematic for an illustrative embodiment of a browser-wallet in accordance with one example of the invention, and how its core functions can be split across different components of the application.

[0054] FIG. 16 provides a diagram illustrating how searching for content can be performed within the infrastructure of an embodiment of the invention.

[0055] FIG. 17 shows an illustrative interaction between local- and global-full copy peers in accordance with an embodiment of the invention.

[0056] FIG. 18 shows a Metanet tree (or graph) for use in reference to the illustrative use case described below.

[0057] FIG. 19 shows a flow chart which illustrates the process embodied by the illustrative use case provided below.

[0058] FIG. 20 is a schematic diagram illustrates a computing environment in which various embodiments can be implemented.

[0059] The term "bitcoin" is used herein for convenience only, and is intended to include all cryptocurrency/blockchain protocols including, but not limited to, all variations that are derived from the Bitcoin protocol as well as any alternative protocols for other blockchains. In the remainder of this document, the protocol determining operation of the embodiments of the present invention will be referred to as the "Metanet protocol". The terms "content" and "data" may be used interchangeably herein to refer to data that is stored in a blockchain transaction in accordance with embodiments of the present invention.

Overview

[0060] As stated above, there is a recognised need for an improved and/or alternative infrastructure for storing, writing, accessing and reviewing data between and by computing nodes. It would be advantageous to use the benefits which are inherent with blockchain technology (e.g. immutable records, cryptographically enforced control and access, built-in payment mechanism, ability to publicly inspect the ledger, distributed architecture etc). However, the construction of a "blockchain implemented internet" is challenging from a number of technical perspectives.

[0061] These challenges may include, but are not limited to: how to locate a particular portion of data within the network; how to secure and control access to the data so that only authorised parties may gain access; how to transfer the data from one party to another in a per-to-peer manner; how to arrange the data so that it can be logically associated yet stored in different locations within the network and how to

subsequently combine it from different locations to provide a collective and augmented result; how to provide and/or store data in a hierarchical fashion; how to allow users and parties with different computing platforms access to the desired data; how to store, provide and share data across a (potentially global) computing network without reliance on or the need for large storage servers and centralised data controllers, and how to improve the efficiency of such data-related activities on the network.

[0062] Herein, "sharing" may include providing to a node or user, sending, communicating, transmitting or providing access to the portion of data. The term "processing" may be interpreted as meaning any activity relating to the transaction or its associated data, including generating, transmitting, validating, accessing, searching for, sharing submitting to a blockchain network, and/or identifying.

[0063] The present invention provides such an improved solution in a manner which, in some ways, is analogous to the internet but achieves its results in an entirely different way, using an entirely different platform of hardware and software components from that known in the prior art. In accordance with embodiments of the present invention, the servers which store internet/web data and provide it to end users are replaced by blockchain transactions residing on the blockchain network. In order to achieve this, several innovations have had to be devised. These are described in the following sections.

[0064] Inserting Data into the Blockchain "Metanet" Referring to FIG. 1, a blockchain transaction embodying the present invention is shown in which first data to be stored on the blockchain is stored in one or more first outputs of the transaction, and second data representing attributes of the first data is stored in one or more second outputs of the transaction. One or more first parts <Content 1> of the first data are stored in a spendable output of the transaction. Data <Attribute 1> and <Attribute 2>, representing respective attributes of the first data, together with a flag indicating that data is being stored according to the Metanet protocol, is stored in a second, unspendable output of the transaction. The term "unspendable" is used to indicate that at least one first and/or second output of the transaction may include a script opcode (OP RETURN) for marking an output as invalid for subsequent use as an input to a subsequent transaction.

[0065] It is advantageous to store the content and attributes parts of the data separately in separate outputs of the transaction.

[0066] FIG. 2 shows a blockchain transaction embodying the present invention in which first data <Content 1> to be stored on the blockchain is stored in an input of the transaction. The Metanet flag, and the attributes data <Attribute 1> and <Attribute 2> are stored in an unspendable output of the transaction, in a manner similar to the arrangement shown in FIG. 1.

[0067] Data Insertion

[0068] Data Insertion Methods

[0069] It is desired to be able to insert the following data into the blockchain

[0070] a) Metanet Flag

[0071] b) Attributes

[0072] c) Content

[0073] The content is data to be stored on the blockchain, the Metanet Flag is a 4-byte prefix that acts as the identifier for any data pertaining to the Metanet protocol, while the

attributes contain indexing, permissioning and encoding information about the content. This could include, but is not limited to, data type, encryption and/or compression schemes. Such attributes are also often referred to as meta-data. Use of this term in the present document will be avoided in order to avoid confusion with transaction meta-data.

[0074] The following techniques can be used to embed this data within a Bitcoin script:

[0075] 1. **OP_RETURN**—In this method all data (attributes and content) are placed after **OP_RETURN** in the locking script of a provably unspendable transaction output.

[0076] An example of an output script using this operator is:

[0077] `UTXO0:           OP_RETURN           <Metanet Flag><attributes><content>`

[0078] 2. **OP_RETURN** with **OP_DROP**—In this case the **OP_RETURN** contains the attributes, whilst the content is stored before an **OP_DROP** in a spendable transaction script (either locking or unlocking). The content can be split into multiple data packets within the transaction input and outputs. However, it is advantageous to insert data into transaction outputs as it is only output scripts that may be signed in the Bitcoin protocol. If the data is inserted into the transaction input, **OP_MOD** can be used as a checksum on the data to ensure its validity in place of miner validation. For example, one could perform a 32-bit **OP_MOD** operation and check that it is equal to a pre-calculated value.

[0079] In this case, the attributes may contain information about how the content data packets are recombined. In addition, providing the hash of the recombined data packets $H(\text{content1} + \text{content2})$ as an attribute enables verification that the recommended recombination scheme has been used.

[0080] A transaction that implements the second data insertion method is shown in FIG. 1. For simplicity this transaction only includes content inserted in its outputs, that is signed by its single input. Content inserted into additional inputs would also be possible using **OP_DROP** statements using this method as shown in FIG. 2.

[0081] If the content is very large it may be advantageous to split it over multiple transactions. Such an arrangement is shown in FIG. 3. FIG. 3 shows a pair of blockchain transactions embodying the present invention in which first data <Content> to be stored on the blockchain is split into two chunks <Content chunk 1> and <Content chunk 2> which can subsequently be recombined as <Content>=<Content chunk 1>||<Content chunk 2>, where the operator '||' concatenates the two chunks of content data. This concatenation operator may be replaced by any desired bitwise or similar piecewise binary operator. The two chunks <Content chunk 1> and <Content chunk 2> are then stored in respective spendable outputs of the separate blockchain transactions, while data relating to the attributes of the content data is stored in respective unspendable outputs of the blockchain transactions. Again, the attributes can contain information about the recombination scheme. For example, the content may be raw data, an executable program or a HTML webpage. Additionally, content1 may include a pointer to the location on the blockchain of content2, which functions in the same way as an embedded HTML link within a webpage.

[0082] It should be noted that both transactions take the same public key P (and ECDSA signature) as input, such that <Content chunk 1> and <Content chunk 2> can be related by the same public key P despite being stored in different transactions with TxID1 and TxID2 respectively.

[0083] Exploiting the Role of Miner Validation

[0084] Here the transaction validation process performed by miners is used to gain an advantage when storing this data. This is because all data in transaction outputs will be signed by the owner of a public key P in at least one transaction input (if the SIGHASH1ALL flag is present) and this signature will be checked in the transaction validation process that all miners perform.

[0085] This ensures

[0086] Data integrity—If the data is corrupted the CHECKSIG operation will fail.

[0087] Data authenticity—The owner of P has provably witnessed and signed the data.

[0088] This is particularly advantageous for content that is split over multiple transactions as the input signature of P provides a provable link between the split components of the data, as described above with reference to the arrangement is shown in FIG. 3.

[0089] Rabin Signature

[0090] Another way to ensure data authenticity is to use Rabin signatures, which can be used to sign the data itself rather than the whole message. This can be advantageous as the signer does not need to sign every individual transaction in which the data appears, and the signature can be re-used in multiple transactions.

[0091] Rabin signatures can be easily validated in script. These can be incorporated in case (2) above by inserting Rabin signature validation before the **OP_DROP** command, i.e.

[0092] `<content1><Rabin Sig (content1)> FUNC_ CHECKRABSIG OP_DROP <H(P1)>[CheckSig P1]`

[0093] It should be noted that this cannot be done in case (1) above since a script containing **OP_RETURN** fails anyway and so no validation can be achieved.

[0094] Specific Example of Use of Rabin Signature

[0095] Introduction

[0096] Digital signatures are a fundamental part of the Bitcoin protocol. They ensure that any Bitcoin transaction recorded on the blockchain has been authorised by the legitimate holder of the Bitcoin being sent. In a standard Bitcoin P2PKH transaction a transaction message is signed using the elliptic curve digital signature algorithm (ECDSA). However the ECDSA signature is generally applied to the whole transaction.

[0097] There are some use cases of the Bitcoin blockchain where a participant from outside the network may want to provide a signature for arbitrary data types which can then be used by network participants. By using Rabin digital signatures any piece of data can be signed—even if it originates from outside the Bitcoin blockchain and then placed in one or multiple transactions.

[0098] It will now be shown how data can be signed and verified directly in Bitcoin script by utilising the algebraic structures of the Rabin cryptosystem

[0099] Rabin Digital Signatures

[0100] The Rabin Digital Signature Algorithm

[0101] Background Mathematics

[0102] Definition—Integers mod p

[0103] The integers modulo p are defined as the set

$$\mathbb{Z}_p := \{1, 2, \dots, p-1\}$$

[0104] Fermat's Little Theorem

[0105] Let p be a prime number. Then for any integer a the following applies

$$a^{p-1} \equiv 1 \pmod{p}$$

[0106] Euler's Criterion

[0107] Let p be a prime number. r is a quadratic residue mod p if and only if

$$r^{\frac{p-1}{2}} \equiv 1 \pmod{p}$$

[0108] Modular Square Roots (p=3 mod 4)

[0109] Let p be a prime number such that p=3 mod 4. Then for any integer r satisfying Euler's criterion, if a is an integer such that

$$a^2 \equiv r \pmod{p}$$

[0110] Then there exists a solution for a of the form

$$a \equiv \pm r^{\frac{p+1}{4}} \pmod{p}$$

[0111] Chinese Remainder Theorem

[0112] Given pairwise coprime positive integers $n_1, n_2, \dots, n_k$ and arbitrary integers $a_1, a_2, \dots, a_k$, the system of simultaneous congruences

$$x \equiv a_1 \pmod{n_1}$$

$$x \equiv a_2 \pmod{n_2}$$

⋮

$$x \equiv a_k \pmod{n_k}$$

has a unique solution modulo $N=n_1 n_2 \dots n_k$. As a special case of the Chinese Remainder Theorem, it can be shown that

$$x \equiv r \pmod{n_1} \text{ and } x \equiv r \pmod{n_2}$$

if and only if

$$x \equiv r \pmod{n_1 n_2}$$

[0113] The Rabin Digital signature Algorithm

[0114] The Rabin digital signature algorithm can be described as follows:

[0115] For any message m let H be a collision resistant hashing algorithm with k output bits. To generate the keys, choose prime numbers p and q, each with bit-length approximately k/2 such that p=3 mod 4, q=3 mod 4 and compute the product $n=p \cdot q$. The private keys are (p, q) and the public key is $n=p \cdot q$.

[0116] To sign the message, m, the signer chooses a padding U such that $H(m||U)$ satisfies

$$H(m||U)^{\frac{p-1}{2}} \equiv 1 \pmod{p}$$

$$H(m||U)^{\frac{q-1}{2}} \equiv 1 \pmod{q}$$

[0117] The signature, S, is computed using the formula

$$S \equiv \left[\left(p^{q-2} \cdot H(m||U)^{\frac{q+1}{4}} \pmod{q} \right) \cdot p + \left(q^{p-2} \cdot H(m||U)^{\frac{p+1}{4}} \pmod{p} \right) \cdot q \right] \pmod{n}.$$

[0118] The signature for the message m is the pair (S, U). Verification can be simply done by checking that, for a given m, U and S

$$H(m||U) \equiv S^2 \pmod{n} \quad (\text{Equation 1}).$$

[0119] This is true if and only if there exists an integer λ in the range 0, ..., n-1 such that

$$H(m||U) + \lambda \cdot n = S^2 \quad (\text{Equation 2}).$$

[0120] The factor λ may be safely included in the signature, to provide the combination (S, λ , U).

[0121] The advantageous features of the Rabin signature scheme are as follows:

[0122] a) Signature generation is computationally expensive, whereas verification of the signature is computationally easy.

[0123] b) The security of the signature relies only on the hardness of integer factorisation. As a result, Rabin signatures are existentially unforgeable (unlike RSA).

[0124] c) The hash function values $H(m||U)$ must be of a similar magnitude to the public key n.

[0125] Verification in script is straightforward as it only requires squaring a given signature, performing a modular reduction and then checking that the result is equal to $H(m||U)$.

[0126] Rabin Signature Proof

[0127] Let p, q be coprime and $n=p \cdot q$. By the Chinese Remainder Theorem it can be shown that

$$S^2 \equiv H(m||U) \pmod{n}$$

If and only if

$$S^2 \equiv H(m||U) \pmod{p}$$

$$S^2 \equiv H(m||U) \pmod{q}$$

It can be shown that

$$S^2 \equiv H(m||U) \pmod{q}$$

[0128] Using

$$\begin{aligned} S &\equiv \left(\left(p^{q-2} H(m||U)^{\frac{q+1}{4}} \pmod{q} \right) \cdot p + \right. \\ &\quad \left. \left(q^{p-2} H(m||U)^{\frac{p+1}{4}} \pmod{p} \right) \cdot q \right) \pmod{q} \\ &\equiv \left(\left(p^{q-2} H(m||U)^{\frac{q+1}{4}} \pmod{q} \right) p \right) \pmod{q} \\ &\equiv p^{q-2} H(m||U)^{\frac{q+1}{4}} \cdot p \pmod{q} \end{aligned}$$

-continued

$$\begin{aligned} &= (p^{q-1} \bmod q) H(m||U)^{\frac{q+1}{4}} \bmod q \\ &= H(m||U)^{\frac{q+1}{4}} \bmod q \end{aligned}$$

[0129] Therefore

$$S^2 \equiv H(m||U)^{\frac{q+1}{2}} \equiv H(m||U)^{\frac{q-1}{2}} \cdot H(m||U) \equiv H(m||U) \bmod q$$

[0130] Where it has been assumed that $H(m||U)$ satisfies Euler's criterion. By a similar calculation one can also show that

$$S^2 \equiv H(m||U) \bmod p$$

[0131] Rabin Signatures in Bitcoin

[0132] Signature Verification in Script

[0133] A small number of arithmetic and stack manipulation opcodes are required to verify a Rabin signature. Consider a redeem script of the form

[0134] OP_DUP OP_HASH160<H₁₆₀(n)> OP_EQUAL-
VERIFY OP_MUL OP_SWAP OP_2 OP_ROLL
OP_CAT FUNC_HASH3072 OP_ADD OP_SWAP
OP_DUP OP_MUL OP_EQUAL

where n is the public key of the signer. This will evaluate to TRUE if and only if is provided with the input

[0135] <S><U><m><λ><n>

where m is an arbitrary message, and (S, λ, U) is a valid Rabin signature. Alternatively, if the Rabin signature is checked using equation 1 above the redeem script is given by

[0136] OP_DUP OP_HASH160<H₁₆₀(n)> OP_DUP
OP_TOALTSTACK OP_SWAP <roll index> OP_ROLL
OP_CAT FUNC_HASH3072 OP_SWAP OP_MOD
OP_SWAP OP_DUP OP_MUL OP_FROMALTSTACK
OP_MOD OP_EQUAL

[0137] In this case the script will evaluate to TRUE if and only if is provided with the input

[0138] <S><U><m><n>

[0139] In both redeem scripts, use has been made of the 3072-bit hash projection function 'FUNC_HASH3072'. For a given message/padding concatenation the FUNC_HASH3072 hash projection is generated using the script

[0140] OP_SHA256 {OP_2 OP_SPLIT OP_SWAP
OP_SHA256 OP_SWAP} (x11)

[0141] OP_SHA256 OP_SWAP OP_SHA256 {OP_CAT}
(x11)

[0142] Compression of Data

[0143] Internet data consists of JavaScript and common file types such as text files (SML, HTML, etc.), video files (MPEG, M-JPEG etc.), image files (GIF, JPEG etc.) and audio files (AU, WAV, etc.), for example as described in greater detail at https://www.doc.ic.ac.uk/~nd/surprise_97/journal/vol1/mmp/#text. Using the above data insertion techniques, these different data types can also be embedded on the blockchain.

[0144] Larger file sizes can be compressed using one of several existing coding schemes before embedding it on the blockchain. Lossless data compression algorithms such as Run-length and Huffman encoding can be used in several applications, including ZIP files, executable programs, text documents and source code.

[0145] Many different algorithms exist depending on the specific input data. Apple Lossless and Adaptive Transform Acoustic Coding can be used to compress audio files, PNG and TIFF for the compression of Graphics files, while movie files can be compressed using one of many lossless video codecs. Any compression of the data content can be indicated using a flag within the attributes. For example, the flag for the LZW lossless coding scheme in the attributes would be <LZW>.

[0146] Encryption and Paid Decryption

[0147] Encryption of Data

[0148] The owner of the content may choose to protect the content before embedding it on the Blockchain. This ensures that the content cannot be viewed without acquiring the necessary permissions.

[0149] There are many well-established techniques for the encryption of data (either plaintext or other data types). These can be categorised as asymmetric encryption or symmetric encryption.

[0150] Elliptic Curve Cryptography (ECC) is asymmetric as it relies on a public-private key pair. It is one of the most secure cryptosystems and is typically used in cryptocurrencies such as bitcoin. For ECC cryptography the Koblitz algorithm can be used to encrypt data.

[0151] In symmetric schemes a single key is used to both encrypt and decrypt the data. The Advanced Encryption Standard (AES) algorithm is considered one of the most secure symmetric algorithms that is seeded by such a secret, for example as described in greater detail in C. Paar and J. Pelzl, Chapter 4 in "Understanding Cryptography," Springer-Verlag Berlin Heidelberg 2nd Ed., 2010, pp. 87-118.

[0152] When encrypting data stored on a blockchain there are advantages in using the same cryptosystems as the underlying blockchain. In bitcoin this is the secp256k1 conventions for ECC key pairs in asymmetric cryptography, and the SHA-256 hash function in symmetric cryptography. These advantages are:

[0153] The security level of the encryption is the same as the underlying system in which the data is stored on.

[0154] The software architecture needed for storing encrypted data will have a smaller codebase.

[0155] The management of keys in a wallet can be used both for transactions and encryption/decryption.

[0156] It is more efficient as the same keys can be used both for encryption and payment in the cryptocurrency, so fewer keys are required. This also reduces storage space.

[0157] Fewer communication channels may be needed to exchange/purchase the ability to decrypt data.

[0158] There is increased security as keys used for encryption and transactions are the same data structure and so targeted attacks for a specific type of key are mitigated.

[0159] Keys can be purchased using the underlying cryptocurrency.

[0160] For illustrative purposes, how the Koblitz algorithm can be used to encrypt data using ECC is described.

[0161] Koblitz Algorithm

[0162] Given a ECC key pair $P_1 = S_1 \cdot G$ the Koblitz algorithm allows anyone to encrypt a message using the public key P_1 such that only someone who knows the corresponding private key S_1 may decrypt the message.

[0163] Suppose it is desired to encrypt the plaintext message ‘hello world’ using the Koblitz method. This is done character-by-character. The first character ‘h’ is encrypted and decrypted as follows.

[0164] 1. The character ‘h’ is mapped to a point on the secp256k1 curve. This is achieved by using the ASCII conventions to map a plaintext character to an 8-bit number. A point on the curve is then calculated by multiplying the base point G by this number. In the present example ‘h’ is mapped to 104 in ASCII, and the elliptic curve point is given by $P_m=104 \cdot G$.

[0165] 2. The point P_m is then encrypted using the public key P_1 . This is achieved by choosing a random ephemeral key k_0 and calculating the pair of points $C_m=\{k_0 \cdot G, Q\}$ where $Q:=P_m+k_0 \cdot P_1$, which may then be broadcast.

[0166] 3. The owner of the private key S_1 may decrypt the original point by calculating $P_m=Q-S_1 \cdot k_0 \cdot G$. They may then recover the original ASCII number by trial-and-error or by means of a look up table to establish which number x corresponds to $P_m=x \cdot G$.

[0167] Using the Blockchain to Purchase Permissions

[0168] Storing data on a blockchain has the obvious advantage that a payment mechanism is built into the system. Payments can be used to purchase

[0169] Decryption data in order to view/use

[0170] Permission to insert data at a particular address

[0171] In both cases the buyer is using a cryptocurrency, for example Bitcoin, to purchase a secret that grants them permission to do something. This secret may be either a hash preimage or a private key.

[0172] An efficient and secure way to make such a purchase is to use an atomic swap. This keeps secure communication channels to a minimum and ensures that either the seller gets paid and the secret is revealed to the buyer, or neither event occurs.

[0173] Besides payments in cryptocurrencies, it can also be convenient to purchase a permission using an access token. This is a secret value (typically a hash preimage) that the buyer owns that they can use in order to make a purchase. Such tokens may be bought in bulk by the buyer ahead of time, and then activated at the time they would actually like to use the permission.

[0174] How atomic swaps are executed will now be described with reference to FIGS. 4 and 5.

[0175] Atomic Swaps Using Hash Puzzles or Private Key Puzzles

[0176] Suppose Alice is the owner of the secret. This secret may be either a hash preimage of a known hash digest, or the private key of a known public key. Suppose Bob wishes to use Bitcoin to buy this secret from Alice. A mechanism known as an atomic swap is described that enables this transaction to occur. It is atomic in the sense that either Alice gets paid Bitcoin and the secret is revealed to Bob, or neither event occurs.

[0177] The method is as follows:

[0178] Alice owns a private key S_A of a public/private key pair $P_A=S_A \cdot G$ and Bob owns a private key S_B of a public/private key pair $P_B=S_B \cdot G$.

[0179] Alice owns a secret which is either the preimage X of a known hash digest $H(X)$, or the private key S_1 of a known public key $P_1=S_1 \cdot G$.

[0180] They agree on a price in Bitcoin for Alice to sell the secret to Bob.

[0181] Prior to these, Bob must set up a transaction to send Alice the ephemeral key k_0 off-block so that she can calculate r_0 , a component of a digital signature.

[0182] Referring now to FIG. 4,

[0183] 1. Bob transfers Bitcoin to Alice that are locked with the following redeem script

R (written schematically):

[0184] For a hash preimage:

$$R=[\text{Hash Puzzle } H(X)][\text{CheckSig } P_A]$$

This forces the preimage X to be exposed in the input of the redeem script.

[0185] For a private key:

$$R=[\text{Private Key Puzzle } P_1, r_0][\text{CheckSig } P_A]$$

This forces the private key S_1 to be able to be calculated from the input to the redeem script. In this case Bob and Alice must agree on an ephemeral key k_0 that is used to construct r_0 , where $(r_0, R_y)=k_0 \cdot G$.

[0186] 2. Since Alice knows her secret (either X or S_1) she can spend her funds on the Bitcoin blockchain by means of the transaction shown in FIG. 5. This allows Bob to determine her secret.

[0187] As an optional security feature, Alice and Bob may use their public keys P_A, P_B to establish a shared secret S known only to both parties. This could be achieved in a manner outlined in International Patent Publication no. WO 2017/145016. In this case S may be added to the preimage X in the hash puzzle in order for X not to be revealed publicly on the blockchain. Similarly, in the private key puzzle S may be used as the ephemeral key k_0 to ensure that only Alice or Bob is able to calculate the private key.

[0188] A time-locked refund can be introduced to the procedure to prevent Bob’s funds being locked by Alice, should Alice decide not to spend her funds.

[0189] Purchase Using Tokens

[0190] Suppose that the same situation exists as described above, but instead paying in cryptocurrency for Alice’s secret at its point of use Bob would like to redeem an access token—that he has bought ahead of time—in exchange for the secret.

[0191] The procedure that Alice and Bob must follow is similar to the case described in the previous section, but instead uses a sequence of similar atomic swaps. There are two phases of the process; token issuance and token redemption.

[0192] Phase 1: Token Issuance

[0193] The token issuance phase is effectively a one-time purchase of tokens by Bob. For example, we will consider the scenario where Alice has 10 distinct secrets $X_1, X_2, \dots, X_{10}$ and Bob wishes to make a single purchase for 10 tokens $T_1, T_2, \dots, T_{10}$ which each grant him access to a respective secret.

[0194] First Bob generates a set of 10 tokens from a secret seed value Y known only to him. These tokens are created by sequential hashing of the seed to form a hash chain, where each token is calculated as

$$T_i=H^{10-i}(Y) \text{ for } i \in \{1, 2, \dots, 10\}.$$

[0195] Alice and Bob now have 10 secret values each, which can be revealed in hash puzzles for example, for the redemption of tokens. In order to issue these tokens however, they must also generate a secret initialising value I_{Alice} and I_{Bob} respectively. These are given as

$$I_{Alice}=k, k \in \mathbb{Z}_{256},$$

$$I_{Bob}=H^{10}(Y).$$

[0196] It should be noted that Alice's initialiser is simply a random integer with no specific meaning, but Bob's initialiser should be the hash of his first token $T_1=H^9(Y)$. Extending the chain of tokens to the initialiser in this way allows the token issuance to also define the token to be used later for successive redemptions. In total the secret values kept by each participant are shown in FIG. 6.

[0197] Now Alice and Bob can agree to a price of 10 cryptocurrency units for the purchase of 10 tokens. The purchase of these tokens can happen in a number of ways, this is illustrated here using an atomic swap. The atomic swap is initiated by Alice and Bob broadcasting the transactions shown in FIGS. 7 and 8 respectively, in both of which the outputs require the solution to two hash puzzles and a valid signature.

[0198] Once both transactions appear in the blockchain Alice and Bob can share their share initialiser values I_{Alice} and I_{Bob} and complete the atomic swap for token issuance. As a result of this atomic swap Alice receives payment for the purchase of 10 tokens and both initialiser secrets are revealed. It should be noted that only Bob's secret $I_{Bob}=H^{10}(Y)$ is meaningful here because it will define the first hash puzzle to be solved [Hash Puzzle (T_1)], whose solution is the preimage $H^9(Y)$ of the initialiser $H^{10}(Y)$.

[0199] Phase 2: Token Redemption

[0200] At some point in the future Bob wants to redeem his first token $T_1=H^9(Y)$ and receive his first secret X_1 , but it will be recalled that he has already paid for this secret by purchasing a valid token. The process of redeeming a token will take the form of another atomic swap, where the solutions to the locking hash puzzles are a token T_i and corresponding secret X_i .

[0201] To redeem his token, Bob should broadcast the transaction shown in FIG. 9, whose output is locked with the two hash puzzles. When Alice sees this transaction she broadcasts her own similar transaction as shown in FIG. 10, whose output is locked with the same two hash puzzles. The two participants can now exchange their secrets T_1 and X_1 and unlock the outputs to these transactions. Both parties can now redeem the nominal fee x by providing the correct unlocking script that also exposes both secrets. The transactions with these unlocking scripts are shown in FIGS. 11 and 12.

[0202] The completion of this atomic swap for redeeming a token reveals Alice's first secret X_1 to Bob, reveals Bob's first token T_1 to Alice and has a net-zero exchange of cryptocurrency funds, given that the amount x is suitably large to encourage both parties to spend the locked outputs. Crucially, this also establishes that the next token Bob can use must be the solution T_2 to the hash puzzle [Hash Puzzle $H(T_2)$], where the target hash $H(T_2)=T_1$ has just been revealed to Alice. This process can be repeated recursively until Bob has used his final token $T_{10}=Y$.

[0203] Naming and Addressing

[0204] Node and Edge Structure

[0205] It has been explained above how data can be inserted into the blockchain by providing it within transactions. We now present a protocol for structuring these transactions in logical way that allows for addressing of

nodes, permissions, and content version control. The structure of this distributed peer metanet is analogous to the existing internet.

[0206] It should be noted that this is a "tier-2" protocol that does not modify the protocol or consensus rules of the underlying blockchain.

[0207] The aim of the structure described here is to

[0208] (i) Associate related content in different transactions to enable searching, identification and access to the data

[0209] (ii) Allow identification of content using human-readable keyword searches, to improve speed, accuracy and efficiency of searching

[0210] (iii) Build and emulate server-like structures within a blockchain

[0211] Our approach is to structure data associated with the Metanet as a directed graph. The nodes and edges of this graph correspond to:

[0212] Node—A transaction associated with the Metanet protocol. A node stores content. (The terms "content" and "data" may be used interchangeably within this document).

[0213] A node is created by including an OP_RETURN that is immediately followed by <Metanet Flag>. Each node is assigned a public key P_{node} . The combination of public key and transaction ID uniquely specify the index of the node $ID_{node} = H(P_{node} || TxID_{node})$.

[0214] The hash function used should be consistent with the underlying blockchain protocol that the invention is to be used with e.g. SHA-256 or RIPEMD-160 for Bitcoin.

[0215] Edge—An association of a child node with a parent node.

[0216] An edge is created when a signature $Sig P_{parent}$ appears in the input of a Metanet transaction, and therefore only a parent can give permission to create an edge. All nodes may have at most one parent, and a parent node may have an arbitrary number of children. In the language of graph theory the indegree of each node is at most 1, and the outdegree of each node is arbitrary.

[0217] It should be noted that an edge is an aspect of the Metanet protocol and is not itself a transaction associated with the underlying blockchain.

[0218] A valid Metanet node (with parent) is given by a transaction of the following form:

TxID _{node}	
Input	Output
<Sig P_{parent} > < P_{parent} >	OP_RETURN <Metanet Flag> < P_{node} > <TxID _{parent} >

[0219] This transaction contains all the information needed to specify the index of the node and its parent

$$ID_{node} = H(P_{node} || TxID_{node}), ID_{parent} = H(P_{parent} || TxID_{parent}).$$

[0220] Moreover, since the signature of the parent node is required, only a parent can create an edge to a child. If the <TxID_{parent}> field is not present, or it does not point to a valid Metanet transaction, then the node is an orphan. It has no higher-level node by which it can be reached.

[0221] Additional attributes may be added to each node. These may include flags, names and keywords. These are discussed later in the this document.

[0222] As shown, the index of a node (transaction) can be broken down into

[0223] a) Public Key P_{node} , which we interpret as the address of the node

[0224] b) Transaction ID $TxID_{node}$, which we interpret as the version of the node

[0225] Two advantageous features arise from this structuring:

[0226] 1. Version control—If there are two nodes with the same public key, then we interpret the node with transaction ID with greatest proof of work as the latest version of that node. If the nodes are in different blocks, then this can be checked with the block height. For transactions in the same block, this is determined by the Topological Transaction Ordering Rule (TTOR).

[0227] 2. Permissioning—A child of a node can only be created if the owner of the public key P_{node} signs the transaction input in the creation of a child node. Therefore P_{node} not only represents the address of a node but also the permission to create a child node. This is intentionally analogous to a standard bitcoin transaction—a public key in not only an address but also the permission associated with that address.

[0228] Note that since the signature of the parent node appears in a UTXO unlocking script it is validated through the standard miner validation process at the point when the transaction is accepted to the network. This means that the permission to create a child node is validated by the bitcoin network itself.

[0229] It is worth noting that standard Internet Protocol (IP) addresses are unique only within a network at a certain point in time. On the other hand, the index of a node in the Metanet is unique for all time and there is no notion of separate networks, which allows data to be permanently anchored to a single object ID ID_{node} .

[0230] The node and edge structure allow us to visualise the Metanet as a graph, as shown in FIG. 13.

[0231] Domains, Naming and Locating Content within the Metanet

[0232] The hierarchy of the Metanet graph allows rich domain-like structure to emerge. We interpret an orphan node as a top-level domain (TLD), a child of an orphan node as a sub-domain, a grandchild as a sub-sub-domain etc., and a childless node as an end-point. See FIG. 13.

[0233] The domain name is interpreted as ID_{node} . Each top-level domain in the Metanet may be thought of as a tree with the root being the orphan node and the leaves being the childless nodes. The Metanet itself is a global collection of trees which form a graph.

[0234] The Metanet protocol does not stipulate that any node contains content data, but leaf (childless) nodes represent the end of a directed path on the data tree, and thus will be used generally to store content data. However, content may be stored at any node in the tree. Protocol-specific flags, included in nodes as attributes, may be used to specify the role of nodes in a data tree (disk space, folders, files or permissioning changes).

[0235] Recall that the internet uses the Domain Name System (DNS) to associate human-readable names to Internet Protocol (IP) addresses. The DNS is in a sense decentralised, although in practice it is controlled by a small

number of key players, such as governments and large corporations. Depending on your DNS provider the same name may take you to different addresses. This issue is inherent when mapping short human-readable names to computer generated numbers.

[0236] We assume that an equivalent distributed system exists that maps a human-readable top-level domain name to the decentralised index ID_{root} of a root node. In other words, there exists a 1-1 function κ that maps human-readable names to Metanet root node indexes, for example

$$\kappa('bobsblog') = ID_{bobsblog} (=H(P_{bobsblog} \| TxID_{bobsblog})).$$

[0237] The input to the left-hand-side is human-readable word, whereas the output on the right-hand-side is a hash digest, which will typically be a 256-bit data structure. Note that $P_{bobsblog}$ and $TxID_{bobsblog}$ are also not human readable in general. In the standard IP protocol this would be a map from www.bobsblog.com to the IP address of the corresponding domain within the network.

[0238] The map κ should be interpreted as a measure to ensure backwards-compatibility of the Metanet with the internet in replicating the human-readability of DNS-issued domain names, but the naming and addressing scheme that provides the structure of the Metanet is not explicitly dependent on this map.

[0239] Possible existing forms of the mapping function κ include the DNSLink system employed by Interplanetary File System (IPFS) or the OpenNIC service (<https://www.openic.org>). This mapping can be stored in an existing TXT record as part of the DNS. This is similar to a DNSLink in the IPFS—see <https://docs.ipfs.io/guides/concepts/dnslink/>. However, in general these sacrifice some element of decentralisation in order to provide a map that is 1-1—see <https://hackernoon.com/ten-terrible-attempts-to-make-the-inter-planetary-file-system-human-friendly-e4e95df0c6fa>

[0240] Vanity Addresses

[0241] The public key used as the address of a Metanet node is not a human-readable object. This can make searching, referencing and inputting activities error prone and slow for human users. However, it is possible to create human-recognisable public key addresses—vanity addresses P_{vanity} —which include a plaintext prefix that can be interpreted directly by a user. Vanity addresses are known in the prior art.

[0242] The difficulty in creating such an address depends on the character length of the desired prefix. This means that human-recognisable vanity addresses may be used as node addresses that rely only on the effort of the owner to create rather than on central issuance. For a given prefix there exist many distinct vanity addresses, due to the characters remaining in the suffix, and hence many node addresses can share a common prefix whilst still retaining uniqueness.

[0243] An example of a vanity address with a desirable prefix is

[0244] $P_{bobsblog}$:
bobsblogHtKNgkdXEobR76b53LETpyT

[0245] Prefix: bobsblog

[0246] Suffix: HtKNgkdXEobR76b53LETpyT

[0247] The vanity address above may be used to sense check the map from the name 'bobsblog' to the node index $ID_{bobsblog}$ and to aid the searchability of Metanet nodes by address. Note that the prefix is not unique here but the entire address itself is a unique entity.

[0248] The combination of a chosen address P_{vanity} with a TxID that together form ID_{node} is also beneficial as it means there is no central issuer of domain names (TxIDs are generated by decentralised proof-of-work) and the names are recoverable from the blockchain itself. Advantageously, there are no longer the points of failure that exist within the internet DNS.

[0249] Since Metanet domains already provide a permissions system (the public key) there is no need to issue a certificate to prove ownership. The use of a blockchain for this purpose has already been explored in namecoin (<https://namecoin.org/>) for example. In accordance with the present invention, however, there is no need to use a separate blockchain for this function as everything is achieved within one blockchain.

[0250] This significantly reduces the amount of resources (hardware, processing resources and energy) required by the invention in comparison to the prior art. It also provides an entirely different architecture in terms of apparatus and arrangement of system components.

[0251] An advantage of this naming system is that a user is able to identify a top-level domain in the Metanet by a memorable word (for example a company name) rather than a hash digest. This also makes searching for the domain faster as it is quicker to search for a keyword rather than a hash digest. It also reduces input errors, thus providing an improved searching tool for blockchain-stored data.

[0252] Given that we have a map from a domain name to a node index we can build up a resource locator similar to that of a Uniform Resource Locator (URL) for the internet. We will call this a Metanet URL (MURL), and takes the form

$$MURL = 'mnp:' + '//domain\ name' + '/path' + '/file'.$$

[0253] Each of the components of a URL—protocol, domain name, path and file—have been mapped to the structure of a MURL, making the object more user-intuitive and enabling it to be integrated with the existing structure of the internet.

[0254] This assumes that each node has a name associated with its public key (address) that is unique at the level within the domain tree. This name is always the right-most component of the MURL for a given node. If two nodes at the same level in the tree have the same name then they will have the same public key and so the latest version is taken.

[0255] The following table gives an analogy between the Metanet protocol and the Internet Protocol:

TABLE	
Summary of the analogies between the Internet and Metanet Protocols.	
Internet	Metanet
Website/file	Node
Owner	Public Key
IP Address (non-unique)	P_{node} Node index (unique)
Domain structure	$ID_{node} = H(P_{node} TxID_{node})$ Node tree structure
Domain Name System	Map from root node name to node index
URL	MURL
http://www.bobsblog.com/path/file	mnp://bobsblog/path/file

[0256] Searching the Metanet

[0257] We have defined an illustrative embodiment the Metanet graph structure such that each node has a unique index and may have a name attributed to it. This allows for content to be located using a MURL. In order to also enable quick search functionality, we allow for additional keywords to be attributed to a node.

[0258] The fixed attributes of a node are the index and index of parent node, and the optional attributes are the name and keywords.

Node attributes	
{	
index:	$H(P_{node} TxID_{node});$
index of parent:	$H(P_{parent} TxID_{parent});$ (NULL if orphan)
name:	'bobsblog';
kwd1:	'travel';
kwd2:	'barbados';
.	.
.	.
}	

[0259] In one example, a practical method for searching the Metanet may be to first use a block explorer to trawl through the blockchain and identify all transactions with the Metanet flag, check that they are valid Metanet nodes, and if so record their indexes and keyword in a database or other storage resource. This database can then be used to efficiently search for nodes with desired keywords. Once the index of the node(s) with the desired keywords is found its content can be retrieved from the block explorer and viewed.

[0260] By way of example, consider the P_1 branch of FIG. 14, where the nodes corresponding to public keys P_0 , P_1 and $P_{1,1}$ represent a home page, topic page and sub-topic page respectively. These nodes are given the names 'bobsblog', 'summer' and 'caribbean', and their attributes are shown below

Home page node P_0	
MURL: mnp://bobsblog	
{	
index:	$H(P_0 TxID_0);$
index of parent:	NULL
name:	'bobsblog';
kwd1:	'travel';
kwd2:	'barbados';
.	.
.	.
}	
Topic page node P_1	
MURL: mnp://bobsblog/summer	
{	
index:	$H(P_1 TxID_1);$
index of parent:	$H(P_0 TxID_0);$
name:	'summer';
kwd1:	'travel';
kwd2:	'barbados';
.	.
.	.
}	
Sub-topic page node $P_{1,1}$	
MURL: mnp://bobsblog/summer/caribbean	
{	
index:	$H(P_{1,1} TxID_{1,1});$
index of parent:	$H(P_1 TxID_1);$
name:	'caribbean';

-continued

kwd1:	'travel';
kwd2:	'barbados';
.	.
.	.
}	

[0261] In this example, the leaf nodes $P_{1,1,1}$, $P_{1,1,2}$ and $P_{1,1,3}$ are given the names 'beaches', 'nightlife' and 'food' respectively and are used to store separate blog posts. The full domain structure is shown on the diagram overleaf, including the MURL search path pertaining to each node in the tree.

[0262] We note that the Metanet can also incorporate a content addressable network (CAN) by storing a hash of the content stored by a node transaction as an additional attribute. This means Metanet nodes may also be indexed and searched for by content hash.

[0263] The naming and addressing methods described above provide numerous technical advantages over the prior art, including:

[0264] 1. Public key addresses—The system uses the same public-private key pairs as the blockchain to assign node addresses. This means that the same set of keys are used for both management of cryptocurrency funds and permissioning of content data. This provides an efficient and secure solution.

[0265] 2. Decentralised domains—The issuance of domain names is fully decentralised through the inclusion of the $TxID_{node}$, which can only be generated by proof of work. The domain names can also incorporate human-recognisable public keys P_{vanity} (vanity addresses) which enable the fair distribution of desired domain public keys. Again, this solution provides enhanced efficiency and security.

[0266] 3. Graph structure—The naming and addressing architecture specifies a graph that can be constructed from the subset of blockchain data comprising Metanet nodes. This design maps the complexity of the internet to the blockchain using an ordered structure such that it fully replicates its functionality and scalability whilst remaining secure.

[0267] Browser-Wallet Application

[0268] Recall that in the Metanet protocol all data lives directly on the blockchain itself. In this section we present embodiments of an illustrative computer application, which we will refer to herein for convenience only as a "browser-wallet", that can efficiently access, display and interact with Metanet data stored on the blockchain.

[0269] We will begin with a discussion of the core components and functionalities of how the browser-wallet interfaces with the distributed peer internet, before providing a more detailed description in the remainder of this section.

[0270] Overview

[0271] Components

[0272] The browser-wallet is an application intended to allow an end-user to interact with the Metanet infrastructure on the blockchain. This application should allow explorative searches of the Metanet graph for specific content embedded in trees. Additionally, the browser-wallet will handle retrieval, decryption, recombination and caching (optional) of content.

[0273] The browser-wallet application will combine these elements with cryptocurrency payment mechanisms by supporting a native (or external) wallet. The browser-wallet will comprise the following core elements, combined into a single computer application.

[0274] Blockchain search engine—Support for a third-party search engine to query Metanet nodes by a variety of indexes including ID_{node} , node name, keywords, block height and TxID.

[0275] Display window—Software that unpacks content returned to the browser by a full-copy blockchain peer. This covers decryption, recombination, caching and redemption of access tokens.

[0276] Cryptocurrency wallet—Dedicated key-management for currency of the blockchain. Can be native to the application or authorised to communicate and synchronise with external wallet (software or hardware). Able to write standard blockchain transactions as well as new Metanet node transactions. Can mediate on-chain purchase of access keys and access tokens.

[0277] Hierarchical, deterministic key management is leveraged for both cryptocurrency public keys and Metanet node addresses.

[0278] Access key/token wallet—Dedicated key-management for purchased access keys or tokens. Can receive purchased keys or tokens using cryptocurrency wallet but has no permissions over them. They may be hidden to the user to allow later expiry. This may be achieved through the use of a trusted execution environment. Timed-access can be secured by synchronising with the blockchain and querying current block height.

[0279] Functionality

[0280] The specification for the Metanet browser-wallet ensures the following functionalities of the application.

[0281] 1. Hierarchical key-management—The keys used for controlling funds and managing Metanet trees (graphs) utilise the same hierarchical deterministic key infrastructure, reducing the burden on users to maintain key records for their Metanet content.

[0282] 2. Pointing to an external cryptocurrency wallet—Ability to authorise and synchronise with an external (non-native to application) wallet allows for additional security by removing the browser-wallet as a point of failure. The application can write blockchain transactions and require the signature of an external wallet that houses keys, delegating this responsibility to separate software or hardware.

[0283] 3. Searching of Metanet content—The browser-wallet can support and query a third-party search engine whose functions may comprise crawling, indexing, servicing and ranking Metanet node transaction data in a global database. A database of OP_RETURN transactions containing the Metanet protocol flag may be constructed. See BitDB 2.0—<https://bitdb.network/>.

[0284] The search engine can serve the browser-wallet with a node index, which allows data to be found.

[0285] 4. Reading and writing data to blockchain—In addition to using search engines and full-nodes to serve the browser with content, the support for a cryptocurrency wallet also allows for content to be written into the Metanet directly from the browser-wallet.

[0286] 5. Decompression and decryption of data—The browser-wallet handles decryption keys and can perform decompression on Metanet content in-situ.

[0287] 6. Caching node identities (ID_{node})—Unique node identities can be cached locally for more efficient lookup and querying.

[0288] 7. Bypassing web-servers—Given a node index, the browser-wallet can query any full-copy member of the peer-to-peer (P2P) blockchain network for the content located at a node. Because the Metanet lives on-chain, any full-copy peer must have a local copy of the node and its content.

[0289] This means that the user's browser-wallet need only query a single peer, which can be done directly and without the need for an intermediary web-server.

[0290] FIG. 15 shows the schematic for the browser-wallet and how its core functions are split across different components of the application.

[0291] Blockchain Search Engine

[0292] Search Engines—Existing Technologies

[0293] Search engines (SEs) as known in the prior art rely on powerful web crawlers to locate, index and rank web content according to the user queries. (The same underlying principles can be extended to a third-party Blockchain SE that crawls the Metanet).

[0294] SEs identify relevant HTML metatags and content through a search of the keywords in the query. The results of the crawl are subsequently indexed where any embedded images/videos/media files are analysed and catalogued. The most relevant results from the index are then ranked programmatically, taking into consideration the user's location, language and device.

[0295] A typical SE should have the following functionality:

[0296] 1. Crawling—Identify internet data and crawl through the relevant metadata, such as domain names, linked pages and related keywords. New internet content is discovered through existing content and also crawled for any relevant information.

[0297] 2. Indexing—Content data are analysed and catalogued. This information is stored in the database.

[0298] 3. Servicing and ranking—Content indexes are ranked in order of relevance to the users' queries.

[0299] Block Explorers

[0300] The closest blockchain analogue to an internet search engine (SE) is a blockchain explorer, sometimes referred to as a 'block explorer' or 'blockchain browser'. Blockchain explorers are web applications which enable user-friendly queries of a blockchain, at a high level, and function similarly to web browsers but are connected to a blockchain rather than the internet. See https://en.bitcoin.it/wiki/Block_chain_browser.

[0301] In most cases, these explorers allow blocks (indexed by hash of the block header), transactions (indexed by TxID), addresses and unspent transaction outputs (UTXOs) to be taken as inputs and searched for. Many explorers also offer their own application programming interfaces (APIs) for retrieving raw transaction and block data. See <https://blockexplorer.com/api-ref>.

[0302] Block explorers, while varying in their capabilities, are generally useful for cataloguing transactions and displaying their basic information—such as transacted currency values, confirmations and histories of coins and

addresses—in a form that is easy for users to digest. Many explorers, such as Bitcoin.com <https://explorer.bitcoin.com/bch> and Blockchain.com <https://www.blockchain.com/explorer> also allow the individual input and locking scripts for transactions to be viewed, although there are inconsistencies between how these and more advanced sites like Blockchain <https://blockchair.com/> choose to provide this information.

[0303] Recently there have been many extensions of basic blockchain explorers used to run web applications based on blockchain data. These applications, such as Memo.cash <https://memo.cash/protocol> and Matter <https://www.mtrr.app/home> act like block explorers that catalogue and organise blockchain transactions that contain specific protocol identifiers, as well as displaying data encoded within those particular transactions.

[0304] However, there are two important issues with using blockchain explorers that are addressed by embodiments of the present invention:

[0305] 1. Universality—There is currently no industry-wide standard for browsing content data that is stored in transactions. Content data refers to any data that does not pertain to the protocol used for creating and securing the underlying blockchain.

[0306] 2. Keyword searching—Content data stored in transactions needs to be retrievable by human-readable keywords. This is not generally a function of current block explorers as they are used to query the protocol-based properties of transactions, such as block height, TxID and addresses rather than taking keywords as search inputs. (However, some e.g. Blockchain can search for words if they are directly included in the script of the transaction).

[0307] Importantly, the powerful naming and addressing structure of the present invention, as discussed above, facilitates and enables the construction of a more sophisticated blockchain explorer than known in the art.

[0308] Proposed Metanet Search Engine

[0309] The browser-wallet application communicates with a third-party search engine for discovery of node identities (ID_{node}). It is envisaged that such a third-party may provide a powerful and versatile service that replicates the capabilities of existing internet search engines.

[0310] A Metanet search engine third-party maintains a database of all Metanet transactions mined into the blockchain identifiable by the Metanet protocol flag. This database can catalogue all Metanet nodes by a range indexes including ID_{nodes} , node names, key words, TxID and block height.

[0311] There already exist services, such as Bit DB <https://bitdb.network/>, which continuously synchronise with the blockchain and maintain transaction data in a standard database format. The browser-wallet offloads the responsibilities of crawling, indexing, servicing and ranking Metanet transactions to such a third-party and makes a connection to their services when locating content stored on the Metanet graph.

[0312] Efficiency savings may be made by having a database that is dedicated to Metanet data only. Unlike Bit DB this would not store the data associated with all transaction, only those containing the Metanet flag. Certain databases, such as non-relational databases like MongoDB, may be more efficient at storing the graph structure of the Metanet.

This would allow for faster querying, lower storage space, and more efficiently associate related content within Metanet domains.

[0313] FIG. 16 shows how the browser-wallet interacts with a third-party search engine when a user searches for content within the Metanet infrastructure. Importantly, it should be noted that, in contrast with the Internet, no routing is necessary and thus the invention provides important advantages in respect of efficiency, speed, processing and required resources.

[0314] The process is as follows

[0315] 1. The end user inputs keywords into the browser-wallet search bar.

[0316] 2. The browser-wallet sends the keyword query to a third-party SE.

[0317] 3. The SE checks the keywords against its database and returns ID_{node} for any Metanet nodes containing relevant content. The third-party can also return other indexes on each node to the user, as well as providing suggestions for related content.

[0318] 4. The browser-wallet uses the node identity and the domain name associated with it to construct a MURL.

[0319] 5. The browser-wallet requests the content belonging to the specified node from any network peer with a full copy of the blockchain.

[0320] 6. The network peer serves the browser-wallet with the requested content. Because the peer has a copy of the blockchain, they must also have a copy of the content and so only one request is made, and it is never forwarded to other network peers.

[0321] It is emphasised that the third-party SE only has the responsibility of indexing and maintaining records of the attributes of Metanet nodes, while the raw content data stored on the nodes is instead stored by network peers (e.g. full-copy peers, miners, archives) with a full copy of the blockchain.

[0322] Content Display—Metanet Browser

[0323] The browser-wallet application emulates the same front-end capabilities that any typical web-browser should provide. These functions include, but are not limited to:

[0324] 1. Searching—Provide access to a search engine (SE) for locating content.

[0325] 2. Retrieval—Communicate with a server to facilitate the transfer of content using a known protocol e.g. Hypertext Transfer Protocol (HTTP).

[0326] 3. Interpreting—Parsing raw code (e.g. in JavaScript) and executing.

[0327] 4. Rendering—Efficient display of parsed content to be viewed by the end user.

[0328] 5. User interface (UI)—Provide an intuitive interface for a user to interact with content, including action buttons and mechanism for user-input.

[0329] 6. Storage—Local temporary storage capacity for caching internet content, cookies etc., to improve repeated access to content.

[0330] In certain embodiments, the software component of the browser-wallet application responsible for acting as a web-browser is able to perform the above functions on Metanet content embedded in the blockchain that is both searchable (using SEs) and retrievable (from peers) using their attributes.

[0331] Recombination, De-Compression and Decryption

[0332] In accordance with certain embodiments of the invention, the web-browser software component of the browser-wallet application is able to handle all operations that need to be performed on given Metanet content. There are many such operations that need to be performed in general, but we assume that at least the following are executed by the application using the Metanet protocol and infrastructure.

[0333] Recombination—In the case that Metanet content needs to be split up and inserted into multiple separate node transactions, the application will request the content from all relevant nodes and reconstitute the original content. The ordering and structure of the splintered content can be encoded using additional flags in each node's attributes.

[0334] Decompression—Where content data is stored on the blockchain in a compressed form, it should include a flag to indicate to the browser-wallet which standard compression scheme has been used. The application will decompress content according to this flag.

[0335] Decryption—Where content is encrypted a flag should be used to signify the encryption scheme. The application will locate a key from its decryption key wallet (as discussed below) and decrypt content data for use according to the encryption scheme used.

[0336] In performing these operations on content data, flags can be used to signify to the browser-wallet that a given operation needs to be performed. This generalises to any other operation, for which a suitable <operation_flag> can be included as part of the attributes of nodes to which the operation applies.

[0337] Caching

[0338] The caching of local files and cookies is a common and important function of typical web-browsers. The browser-wallet application also uses local storage in a similar way in order to optionally keep a record of ID_{node} and other node attributes that pertain to content of interest. This allows more efficient lookup and retrieval of content from frequently-visited Metanet nodes.

[0339] The Metanet solves the problem inherent with caching internet data that it is mutable and can be changed or censored by web-browsing software depending on the provider. When caching Metanet data a user can always easily verify that it is in the same state as when originally included as an immutable record on the blockchain.

[0340] Cryptocurrency Wallet

[0341] Hierarchical Deterministic Key Management

[0342] Deterministic keys Dk are private keys initialized from a single “seed” key (See Andreas M. Antonopoulos, Chapter 5 in “Mastering Bitcoin” O’Reilly 2nd Ed., 2017, pp. 93-98). The seed is a randomly generated number that acts as a master key. A hash function can be used to combine the seed with other data, such as an index number or “chain code” (see HD Wallets—BIP-32/BIP-44), to derive deterministic keys. These keys are related to one another and are fully recoverable with the seed key. The seed also permits the easy import/export of a wallet between different wallet implementations, giving an additional degree of freedom if the user wishes to use an external wallet in conjunction with the Metanet browser-wallet.

[0343] A hierarchical deterministic (HD) wallet is a well known derivation method of deterministic keys. In HD wallets, a parent key generates a sequence of children keys,

which in turn derive a sequence of grandchildren keys, and so on. This tree-like structure is a powerful mechanism for managing several keys.

[0344] In a preferred embodiment, a HD wallet can be incorporated into the Metanet architecture illustrated in FIG.

16. Advantages of using HD wallets include:

[0345] 1. Structure Additional organizational meaning can be expressed using different branches of subkeys for different purposes. For example, a user could dedicate different branches (and their corresponding subkeys) to different types of data.

[0346] 2. Security Users can create a sequence of public keys without the corresponding private keys, making HD wallets functional in a receive-only capacity and suitable for use on insecure servers. Also, since fewer secrets need to be stored there is a lower risk of exposure.

[0347] 3. Recovery If keys are lost/corrupted then they can be recovered from the seed key.

[0348] Native (Internal) and External Wallet Support

[0349] Advantageously, embodiments of the invention can directly merge the functionality of traditional web-browsers with one or more cryptocurrency wallets. This is fundamentally how the Metanet combines the payment for “internet” content with its delivery to the end user.

[0350] To achieve this, embodiments of the browser-wallet may have a dedicated, in-built software component that operates as a cryptocurrency wallet. This wallet is native the application itself and can be used to manage cryptocurrency private keys and authorise transactions as payment for Metanet content within the browser-wallet itself.

[0351] This means that the browser component of the application can prompt the wallet component to authorise a payment that is required—by purchasing a decryption key, access token or otherwise—to view Metanet content. The application does not need to invoke an external third party to process the payment, and thus the Metanet content of interest is consumed by the application and paid for in-situ.

[0352] External Wallets

[0353] The same advantages and functionality can be achieved by embodiments of the application if a user wishes to instead manage or keep their cryptocurrency private keys on an external wallet (software or hardware) or even use multiple wallets. This may be performed in lieu of, or in conjunction with, the application’s native wallet.

[0354] In such embodiments, the application establishes a link or pairing with an external wallet(s), and synchronises with it, but does not store private keys in the browser-wallet itself. Instead, when the browser component prompts a payment to be made for content, the application requests an authorisation by digital signature from the external wallet of choice. This authorisation is made by the user and the browser-wallet can broadcast the transaction and view the paid content.

[0355] Reading and Writing Metanet Transactions

[0356] An intrinsic advantage of the Metanet is that it uses the same data-structure—the blockchain—to record both payments and content data. This means that software wallets can be used to write content data to the Metanet infrastructure in addition to creating transactions that are purely based on the exchange of cryptocurrency.

[0357] The native wallet built-in to the application is able to write transactions to the blockchain that are more complex than typical simplified payment verification (SPV)

clients—see <https://bitcoin.org/en/glossary/simplified-payment-verification>. The wallet allows users to choose to write a Metanet node transaction to the blockchain directly from the application by selecting content data, from their computer, to be embedded in the blockchain.

[0358] As the browser-wallet application has a user interface (UI) it allows for the wallet component to create and broadcast transactions that include content data that has been constructed either in the browser-component or on the user’s computer beforehand. This capability would be far more difficult to achieve for a purpose-built wallet to handle on its own.

[0359] Access Key/Token Wallet

[0360] Recall from above that, built in to the Metanet protocol, is the ability to encrypt content using an ECC keypair or AES symmetric key, and the ability purchase the corresponding decryption key or token. We will refer to these as access keys or access tokens.

[0361] Such keys/tokens grant the user permission to view or edit content (either single use or multi-instance use) and play a distinct role from keys that control the users cryptocurrency wallet (although the same key may be used for both purposes if desired). For this reason, it is advantageous to introduce a new wallet, separate from the application’s native cryptocurrency wallet, that is used for storing and managing access keys and tokens.

[0362] One can also introduce the notion of timed access to Metanet content by allowing access keys/tokens to be burned after a certain time period. This can be achieved by requiring that access keys/tokens are stored in a Trusted Execution Environment (TEE) and are not directly accessible to the user.

[0363] The fact that access keys/tokens may be “burned” is also a motivating factor for not storing them in the cryptocurrency wallet to ensure there is no risk of cryptocurrency private keys being burned.

[0364] In a similar way to the cryptocurrency wallet, decryption keys and access tokens can be stored and managed deterministically to facilitate efficient handling and deployment. Decryption keys (e.g. ECC private keys) can be generated and recovered by subsequent additions to a master key, while access tokens can be reconstructed using a hash chain that is seeded by some initial token.

[0365] It is important to make the distinction here that the cryptocurrency wallet handles the deterministic key generation for key pairs that are used in transacting with other users and creating new Metanet nodes, whereas a key/token wallet(s) handles keys and tokens that have been purchased by the cryptocurrency wallet.

[0366] Block Height Permissioning

[0367] Timelocks can be included in the bitcoin script language to enable block height permissioning. The op_code OP_CHECKLOCKTIMEVERIFY (CLTV) sets the block height at which a transaction output (UTXO) is permissible for spending.

[0368] The advantages of block height permissioning are twofold:

[0369] 1. Version control—In the Metanet Protocol, the newest version of a node can be identified from the node at the greatest block height. The browser-wallet can be set up to only display the most recent version of a file by block height, allowing proof-of-work version control.

[0370] 2. Timed access—The Browser-wallet application can periodically burn the decryption keys atomically purchased by the user. This ensures that viewers can only access content data during the time period for which they have paid. Cloning of the decryption keys can be prevented by storing them in a trusted execution environment (TEE). Moreover, the atomic swap involves the purchase of a deterministic key Dk (for decryption of the content data). Although this deterministic key is publicly visible, the TEE can be used to sign for the combination of Dk and a securely enclaved private key.

[0371] The browser-wallet can be arranged to synchronise with the current state of the blockchain in order to use block height as its own proxy for time, rather than relying on any external clock or third-party time oracle.

[0372] Bypassing Web-Servers

[0373] The invention allows a new mechanism for a browser (client) and a web server to communicate and exchange information over the distributed peer internet that bypasses domain name system (DNS) servers and typical network routing procedures. See http://www.theshulers.com/whitepapers/internet_whitepaper/. The invention provides a new network architecture, from which the browser-wallet application can be served content, comprising peers that maintain a full-copy of the blockchain.

[0374] Local Full-Copy Peers

[0375] Consider a system of local peers in each geographical area e.g. postcode, town, city. We assume that within this local network at least one peer maintains a full-copy of the blockchain, which we will refer to as a Local Full-Copy Peer (LFCP). For our purposes, an LFCP need only store the blockchain transactions that include the Metanet flag, but we do not limit them to this.

[0376] All users default to sending ‘get’ requests to LFCPs. As the peers maintain a complete and up-to-date copy of the entire blockchain, all requests can be served because any node ID that is queried will be available to the LFCP. Note that a Metanet search engine may also act as an LFCP if the SE is powerful and large enough to both store Metanet content and perform the main functions of a typical SE.

[0377] In the simplest case, every LFCP will have the same storage and disk space overheads as they will all need to be able to store the full blockchain (around 200 GB at the time of writing). The distinction between each LFCP is that they should scale their capability to respond to the demands of local requests from Metanet users. So, if each Metanet user in the world queries their closest LFCP by default, each LFCP should aim to scale in their operational capacity to meet their local demand. Densely-populated areas like cities will require LFCP operations comprising of many clustered servers, while sparser areas like small towns will require smaller LFCP operations.

[0378] It is important to note that the disk space requirement is universal, while the CPU requirement for each LFCP adapts to local network demand. This is an example of an adaptable network, such as Freenet—see <https://blockstack.org/papers/>.

[0379] One advantage of such a system is that users only need to make a single (local) connection to their LFCP when retrieving the content associated with a given ID_{node}. There

is no need for the LFCP to forward the request to other peers as they are guaranteed to be able to serve the required content themselves.

[0380] The Metanet provides many advantages over the internet—such as decentralisation and deduplication—that are similar to other peer-to-peer (P2P) file-sharing services like IFPS. However, the Metanet improves on these existing P2P models by ensuring immutability and, crucially, removing the need to flood the network with requests for given content.

[0381] The Metanet infrastructure is also robust to the compromise of any one LFCP by employing a network of these peers. This means that if a LFCP is disabled then the end user simply defaults to using their next nearest LFCP. This can be made more efficient if LFCPs communicate with each other to indicate which nearby peers are below or above capacity in terms of requests at any given time. This can allow users to send their request to the most appropriate peer and establish dynamic equilibria in the request distribution between nearby LFCPs.

[0382] Global Full-Copy Peers

[0383] We now consider a scenario when the universal disk space requirement becomes too great for smaller peers, which may happen as the Metanet portion of the blockchain scales and grows with adoption.

[0384] In this case smaller LFCPs should use their disk space capacity to store Metanet node transactions based on a popularity system (there are existing techniques for ranking content by request volume and nature). This means that LFCPs now tailor both their CPU (for request handling ability) and their storage allocation (for content-serving ability) to suit their local geographic demands both in volume and nature of content.

[0385] In order to address the fact that LFCP are now unable to store all Metanet transaction content the concept of a Global Full-Copy Peer (GFCP) can be utilised. A GFCP is a full-copy peer that has the following properties:

[0386] 1. A GFCP grows its disk space capacity in order to maintain a full-copy of the blockchain at all times.

[0387] 2. A GFCP has substantial CPU resources such that it can handle significantly more requests than an LFCP. A global full-copy peer should be able to handle a sudden increase in demand should many LFCPs become compromised.

[0388] There are two main functions of a GFCP. First, to act as a failsafe for user requests of Metanet content in times of overflow requests from LFCPs. Second, GFCPs act as archive peers to store all Metanet content mined historically, which ensures that any Metanet node content can be accessed even if many LFCPs omit some content from their local storage provisions.

[0389] Global Data Banks

[0390] The concept of a GFCP is a powerful one and illustrates how the overall architecture of the Metanet provides a solution to an existing problem; that of creating all-encompassing, global data banks.

[0391] Before now, it has not been possible to safely construct a universal and globally-accessible data bank because it would need to be maintained by a central authority. This central authority injects both a point of failure and of trust into the system. Crucially, if one organisation is relied upon to store and maintain all internet data then we need to trust the facts that they are doing so correctly and legitimately, without corrupting the information.

[0392] With the Metanet infrastructure, we have effectively removed both of these problems, those of trust and of centrality, from the concept of a global data centre. Now, such a GFCP can be created because they are only relied upon to provide the required disk space for storage and not to verify and authenticate the information to be stored.

[0393] With the Metanet, the process of verifying what is stored is done by miners and hence universal, global data banks can be trusted because they cannot corrupt blockchain information. The GFCP does not need to be trusted and need only provide storage. The fact that all GFCPs can store the same information, that is always verifiable and provable against the blockchain itself, means that it can be replicated across many such GFCPs. This means we have also solved the issue of having a single point of failure by enabling many global data banks to exist in parallel and provably store the same information.

[0394] FIG. 17 shows a system of two LFCPs and one GFCP and illustrates how each peer can support the other in a network that is robust to the compromise of individual peers.

[0395] The aspects of the invention which can be implemented in embodiments of the browser-wallet application described above provide numerous distinguishing features and advantages over the prior art, including but not limited to:

[0396] 1. Deterministic keys—Hierarchical deterministic key management for both cryptocurrency and Metanet addresses is performed in the same wallet component of the application. This allows organisation of keys with multiple functions that reduces their storage requirements and enables key recovery.

[0397] 2. Payment mechanism—The application allows consumers to pay merchants directly without needing to point to another application or third-party payment service that would conventionally authenticate and provide trust. This allows the purchase and delivery of digital content to take place via the same blockchain platform. The application inherits the advantages of bitcoin payments including low value exchanges or more complex transactions involving multiple parties.

[0398] 3. Bypassing web-servers—The application facilitates the bypassing of traditional web-servers, which would conventionally process a high volume of traffic, requests and routing. This is because the application need only request content from a single LFCP, which is guaranteed to serve the user without needing to forward the request to other LFCPs. This reduces the overall traffic volume as well as the completion time of each request.

[0399] 4. Timed access—The application facilitates timed access to content by synchronising with the blockchain and using it to enforce access permissions based on its current state. This removes the need for third party services to monitor a users' privileges over time while protecting the rights of the original owner.

[0400] Use Case—Decentralised App Store (Swapp Store)

[0401] The first use case presented here (for illustrative purposes only) for the Metanet architecture is for the decentralised payment for and distribution of applications (apps).

[0402] We consider a scenario in which an app developer Alice and a consumer Bob wish to transact with each other. This transaction will take the form of an atomic swap in which money is exchanged for a secret key that grants Bob

access to the application data. The encrypted application data is already public as part of a Metanet node transaction.

[0403] An atomically-swapped application is known as a Swapp. A third-party platform (Swapp store) may be used for cataloguing and advertising applications that exist on the Metanet, but the payment for and transfer of the access key to a user such as Bob does not need to involve any third party and can be done directly between merchant and consumer.

[0404] The following section details a process that may be used for buying and selling Swapps from the creation of an app by Alice to its deployment by Bob. Throughout the process, Alice and Bob will use their respective browser-wallets to interact with the Metanet.

[0405] Publishing

[0406] 1. Alice writes an application. The data that constitutes this application is the content denoted by $\langle \text{App} \rangle$. She also encrypts it $\text{sa}\langle \text{e}(\text{App}) \rangle$ using a secret key S_k .

[0407] 2. Alice creates a node transaction, $\text{ID}_{\text{AliceApp}}$ to set up her first Metanet domain (tree). She generates $1\text{AliceAppHtKNgkdXEobR76b53LETpy}$ (P_{AliceApp}) to be used as the node address.

[0408] 3. Alice then creates children of the first node to form a tree that corresponds to a Metanet library of her applications. Alice's tree domain is shown in FIG. 18.

[0409] One of the leaf nodes on this tree is the node corresponding to her application $\langle \text{App} \rangle$ with index ID_{App} . In this node, Alice inserts the encrypted application data $\langle \text{e}(\text{App}) \rangle$ into the input script (scriptSig) of the node. The app data is encrypted using the Koblitz method using a secret key S_k .

[0410] This node transaction is shown below.

TxID _{App}	
Input	Output
$\langle \text{Sig } P_{\text{Puzzle}} \rangle \langle P_{\text{Puzzle}} \rangle \langle \text{e}(\text{App}) \rangle$	OP_RETURN $\langle \text{Metanet Flag} \rangle \langle P_{\text{App}} \rangle$
OP_DROP	$\langle \text{TxID}_{\text{Puzzle}} \rangle$

[0411] 4. Alice broadcasts $\text{ID}_{\text{AliceApp}}$, P_{AliceApp} and domain name 'AliceApp' publicly. This can be via social media, an internet website or by using a third-party Metanet website.

[0412] Purchasing

[0413] 1. Bob wants to download a puzzle game and sees Alice's app listed on a Metanet website (Swapp store) that he views on his browser-wallet.

[0414] 2. Bob then communicates with Alice, using information from the website, and sets up an atomic swap. The swap is designed such that Bob will pay an agreed price in Bitcoin to Alice and Alice will reveal the secret key S_k or neither event occurs.

[0415] 3. The atomic swap is completed and Bob's browser-wallet stores the secret key S_k in its access key/token wallet.

[0416] Deployment

[0417] Bob now has the key S_k that will allow him to decrypt the application data Alice published previously. In order to download the app and deploy it Bob does the following.

[0418] 1. Bob uses a Metanet search engine (SE) to find the MURL associated with the encrypted app data $\langle \text{e}(\text{App}) \rangle$. He uses the keywords 'AliceApp' and 'App'

as input to the search bar in his browser-wallet. The third-party SE resolves the query and returns the following MURL:

[0419] mnp://aliceapp/games/puzzle/app

[0420] This locator corresponds to the unique Metanet node ID_{App} which includes the encrypted app data in its input script.

[0421] 2. Bob's browser wallet receives this MURL and sends a request to the nearest appropriate LFCP. This peer serves Bob with the requested data $\langle e(App) \rangle$.

[0422] 3. The browser-wallet processes the data according to the attributes of ID_{App} . This includes using the secret key s_k to decrypt the application data and process $\langle App \rangle$.

[0423] 4. Bob downloads the application $\langle App \rangle$ from his browser to his computer. Bob can now deploy the application locally without having to re-purchase access.

[0424] FIG. 19 illustrates the entire process outlined in the above illustrative use case. The flow chart shows two action branches, Alice's (starting on the left hand side) and Bob's (starting on the right hand side). The branch corresponding to Alice shows the initial publishing phase and Bob's shows the phase of setting up the purchase via atomic swap.

[0425] On Bob's branch, he broadcasts the following transaction $TxID_{Bob}$ as the atomic swap set-up phase:

$TxID_{Bob}$			
Inputs		Outputs	
Value	Script	Value	Script
x BCH	$\langle \text{Sig } P_B \rangle \langle P_B \rangle$	x BCH	[Private Key Puzzle P_k , r_0] [CheckSig P_A]

[0426] In this transaction, the output is locked with a private key puzzle that requires the secret decryption key s_k to be revealed to Bob in order for Alice to spend. Alice and Bob's branches in this diagram converge at the point where Alice successfully completes the atomic swap transaction. This is achieved when Alice broadcasts the transaction $TxID_{Alice}$:

$TxID_{Alice}$			
Inputs		Outputs	
Value	Script	Value	Script
x Bitcoin	$\langle \text{Sig } P_A \rangle \langle P_A \rangle \langle \text{Sig } P_1, r_0 \rangle \langle P_1 \rangle$	x Bitcoin	[CheckSig P_A]

[0427] As soon as this transaction is broadcast, Alice and Bob's action branches diverge once more. Alice receives payment of x Bitcoin while Bob receives the secret decryption key s_k and is able to retrieve and decrypt Alice's application from the Metanet.

[0428] Turning now to FIG. 20, there is provided an illustrative, simplified block diagram of a computing device 2600 that may be used to practice at least one embodiment of the present disclosure. In various embodiments, the computing device 2600 may be used to implement any of the systems illustrated and described above. For example, the computing device 2600 may be configured for use as a data

server, a web server, a portable computing device, a personal computer, or any electronic computing device. As shown in FIG. 20, the computing device 2600 may include one or more processors with one or more levels of cache memory and a memory controller (collectively labelled 2602) that can be configured to communicate with a storage subsystem 2606 that includes main memory 2608 and persistent storage 2610. The main memory 2608 can include dynamic random-access memory (DRAM) 2618 and read-only memory (ROM) 2620 as shown. The storage subsystem 2606 and the cache memory 2602 and may be used for storage of information, such as details associated with transactions and blocks as described in the present disclosure. The processor(s) 2602 may be utilized to provide the steps or functionality of any embodiment as described in the present disclosure.

[0429] The processor(s) 2602 can also communicate with one or more user interface input devices 2612, one or more user interface output devices 2614, and a network interface subsystem 2616.

[0430] A bus subsystem 2604 may provide a mechanism for enabling the various components and subsystems of computing device 2600 to communicate with each other as intended.

[0431] Although the bus subsystem 2604 is shown schematically as a single bus, alternative embodiments of the bus subsystem may utilize multiple busses.

[0432] The network interface subsystem 2616 may provide an interface to other computing devices and networks. The network interface subsystem 2616 may serve as an interface for receiving data from, and transmitting data to, other systems from the computing device 2600. For example, the network interface subsystem 2616 may enable a data technician to connect the device to a network such that the data technician may be able to transmit data to the device and receive data from the device while in a remote location, such as a data centre.

[0433] The user interface input devices 2612 may include one or more user input devices such as a keyboard; pointing devices such as an integrated mouse, trackball, touchpad, or graphics tablet; a scanner; a barcode scanner; a touch screen incorporated into the display; audio input devices such as voice recognition systems, microphones; and other types of input devices. In general, use of the term "input device" is intended to include all possible types of devices and mechanisms for inputting information to the computing device 2600.

[0434] The one or more user interface output devices 2614 may include a display subsystem, a printer, or non-visual displays such as audio output devices, etc. The display subsystem may be a cathode ray tube (CRT), a flat-panel device such as a liquid crystal display (LCD), light emitting diode (LED) display, or a projection or other display device. In general, use of the term "output device" is intended to include all possible types of devices and mechanisms for outputting information from the computing device 2600. The one or more user interface output devices 2614 may be used, for example, to present user interfaces to facilitate user interaction with applications performing processes described and variations therein, when such interaction may be appropriate.

[0435] The storage subsystem 2606 may provide a computer-readable storage medium for storing the basic programming and data constructs that may provide the functionality of at least one embodiment of the present

disclosure. The applications (programs, code modules, instructions), when executed by one or more processors, may provide the functionality of one or more embodiments of the present disclosure, and may be stored in the storage subsystem **2606**. These application modules or instructions may be executed by the one or more processors **2602**. The storage subsystem **2606** may additionally provide a repository for storing data used in accordance with the present disclosure. For example, the main memory **2608** and cache memory **2602** can provide volatile storage for program and data. The persistent storage **2610** can provide persistent (non-volatile) storage for program and data and may include flash memory, one or more solid state drives, one or more magnetic hard disk drives, one or more floppy disk drives with associated removable media, one or more optical drives (e.g. CD-ROM or DVD or Blue-Ray) drive with associated removable media, and other like storage media. Such program and data can include programs for carrying out the steps of one or more embodiments as described in the present disclosure as well as data associated with transactions and blocks as described in the present disclosure.

[0436] The computing device **2600** may be of various types, including a portable computer device, tablet computer, a workstation, or any other device described below. Additionally, the computing device **2600** may include another device that may be connected to the computing device **2600** through one or more ports (e.g., USB, a headphone jack, Lightning connector, etc.). The device that may be connected to the computing device **2600** may include a plurality of ports configured to accept fibre-optic connectors. Accordingly, this device may be configured to convert optical signals to electrical signals that may be transmitted through the port connecting the device to the computing device **2600** for processing. Due to the ever-changing nature of computers and networks, the description of the computing device **2600** depicted in FIG. **20** is intended only as a specific example for purposes of illustrating the preferred embodiment of the device. Many other configurations having more or fewer components than the system depicted in FIG. **20** are possible.

[0437] It should be noted that the above-mentioned embodiments illustrate rather than limit the invention, and that those skilled in the art will be capable of designing many alternative embodiments without departing from the scope of the invention as defined by the appended claims. In the claims, any reference signs placed in parentheses shall not be construed as limiting the claims. The word “comprising” and “comprises”, and the like, does not exclude the presence of elements or steps other than those listed in any claim or the specification as a whole. In the present specification, “comprises” means “includes or consists of” and “comprising” means “including or consisting of”. The singular reference of an element does not exclude the plural reference of such elements and vice-versa. The invention may be implemented by means of hardware comprising several distinct elements, and by means of a suitably programmed computer. In a device claim enumerating several means, several of these means may be embodied by one and the same item of hardware. The mere fact that certain measures are recited in mutually different dependent claims does not indicate that a combination of these measures cannot be used to advantage.

1. A method of storing data on a blockchain, the method comprising:

generating a blockchain transaction having at least one first input and or at least one first output containing first data to be stored on the blockchain, and at least one second input and or at least one second output containing second data representing at least one attribute of said first data, wherein at least one said second input and or at least one said second output is separate from the or each said first input and or first output.

2. A method according to claim 1, wherein the first data is contained in a plurality of said first inputs and or said first outputs.

3. A method according to claim 1, wherein said second data includes data relating to recombination of said first data.

4. A method according to claim 1, wherein at least one said second input and or at least one said second output includes indication data indicating that the transaction includes said first data.

5. A method according to claim 1, wherein the second data represents at least one attribute of said first data, being at least one of data type, data encryption scheme, data compression scheme, indexing information, permission information, encoding information, keyword information or searching information of said first data.

6. A method according to claim 1, wherein the first data is contained in at least one said first output.

7. A method according to claim 1, wherein at least one said first and/or second output includes a script opcode for marking an output as invalid for subsequent use as an input to a subsequent transaction.

8. A method according to claim 1, wherein at least one said first output is redeemable by means of at least one key corresponding to a respective digital signature applied to said first data.

9. A method according to claim 1, wherein a digital signature is applied to said first data.

10. A method according to claim 9, wherein at least one said digital signature is based on a cryptography system having a public-private key pair, wherein a private key of the public-private key pair is based on a plurality of prime numbers and a corresponding public key is based on a product of a plurality of said prime numbers.

11. A method according to claim 10, wherein at least one said digital signature is a Rabin signature.

12. A method according to claim 1, further comprising applying data compression to said first data and/or said second data.

13. A computer-implemented system comprising:
a processor; and

memory including executable instructions that, as a result of execution by the processor, causes the system to perform any embodiment of the method as claimed in claim 1.

14. A non-transitory computer-readable storage medium having stored thereon executable instructions that, as a result of being executed by a processor of a computer system, cause the computer system to at least perform an embodiment of the method as claimed in claim 1.

15. A computer-implemented system comprising:
a processor; and

memory including executable instructions that, as a result of execution by the processor, causes the system to perform any embodiment of the method as claimed in claim 2.

16. A computer-implemented system comprising:
a processor; and
memory including executable instructions that, as a result of execution by the processor, causes the system to perform any embodiment of the method as claimed in claim 3.

17. A computer-implemented system comprising:
a processor; and
memory including executable instructions that, as a result of execution by the processor, causes the system to perform any embodiment of the method as claimed in claim 4.

18. A non-transitory computer-readable storage medium having stored thereon executable instructions that, as a result of being executed by a processor of a computer system, cause the computer system to at least perform an embodiment of the method as claimed in claim 2.

19. A non-transitory computer-readable storage medium having stored thereon executable instructions that, as a result of being executed by a processor of a computer system, cause the computer system to at least perform an embodiment of the method as claimed in claim 3.

20. A non-transitory computer-readable storage medium having stored thereon executable instructions that, as a result of being executed by a processor of a computer system, cause the computer system to at least perform an embodiment of the method as claimed in claim 4.

* * * * *